WearTo – Intelligent Travel Style & Shopping Assistant

Group Final Report

NUS-ISS | Intelligent Reasoning Systems

Group 17

A0353874R Agarwal Hritik Kumar

A0353592X Jiang Yushan

A0351286B Lu Yujing

A0314662H Ma Bofei

A0330891A Zhang Yurun

Content

1 Introduction 4

1.1 Overview 4

1.2 Motivation 4

1.3 Project Goal 4

2 Project Background 5

2.1 Market Context 5

2.2 Challenges We Address 5

2.3 Market Landscape 7

3 Business Case and Market Research 8

3.1 Business Context and Problem 8

3.2 Market Trends and User Expectations 8

3.3 Competitive Landscape 8

3.4 Market Opportunity and Business Value 9

4 System Design and Model 10

4.1 Use Cases 10

4.2 User Interaction Flow 11

4.3 System Architecture Overview 13

4.4 Design Summary 16

5 System Development & Implementation 17

5.1 User Requirement Understanding and Contextual Reasoning 17

5.2 Inspiration Module 18

5.3 Product Retrieval, Wardrobe Image Understanding and System Integration 21

5.4 Outfit Compatibility Prediction and Ranking 22

5.5 Constraint Optimisation, Final Recommendation and User Interaction 24

6 Experiments 27

6.1 Requirement Understanding and Contextual Reasoning 27

6.2 Inspiration Module 30

6.3 Product Retrieval, Wardrobe Understanding and System Integration 32

6.4 Outfit Compatibility Model Evaluation 33

6.5 Final Outfit Ranking and Beta Selection 33

7 Findings and Discussion 37
8 Future Work 38
8.1 Systematic Evaluation of the End-to-End System 38
8.2 Contextual Reasoning, Multilingual Support and Personalisation 38
8.3 Fashion Inspiration and Product Retrieval 38
8.4 Outfit Compatibility Modelling 39
8.5 Recommendation, Interaction and System Efficiency 39
Appendix A: Project Proposal 40
A.1 Project Details 40
A.2 Background and Objectives 40
A.3 Proposed Scope 41
A.4 Proposed Reasoning Techniques 41
A.5 Data and Resources 42
A.6 Evaluation Plan 42
A.7 Anticipated Challenges 43
A.8 Data Contract 43
Appendix B: Course Mapping 45
B.1 User Requirement Understanding and Contextual Reasoning 45
B.2 Inspiration Module 45
B.3 Product Retrieval, Wardrobe Image Understanding and System Integration 46
B.4 Outfit Compatibility Prediction and Ranking 47
B.5 Constraint Optimisation, Final Recommendation and User Interaction 47
Appendix C: Installation and User Guide 49
C.1 Requirements 49
C.2 Installation 49
C.3 Launching the Website 50
C.4 User Guide 51
C.5 Troubleshooting 54
References 55

1 Introduction

1.1 Overview

With the ever-growing number of fashion items available online, customers now have access to an increasingly diverse range of product choices. However, choosing suitable and preferred outfits for a specific travel scenario remains a complex decision-making problem. Travel outfits are influenced not only by personal preferences, but also by factors such as the destination, travel duration, weather conditions, planned activities, and budget. Moreover, such information is often scattered across weather forecasts, social media, online shopping platforms, and users' own requirements.

Therefore, in this project we propose to develop a travel-oriented intelligent shopping agent system. The system understands users' requirements through natural language interactions, and combines weather information, planned activities, and destination-related fashion trends for reasoning and retrieval. It then searches for real products from e-commerce platforms and combines them with clothes already owned by the user when available. Candidate outfits are filtered and ranked based on travel suitability, product relevance, hard constraints such as budget, and outfit compatibility, and the system finally recommends suitable outfits for the specific trip. The focus of this project is to explore how to transform unstructured travel needs and external information into structured outfit candidates and final purchasing decisions, providing more feasible and actionable decision support for complex, multi-constraint travel fashion shopping scenarios.

1.2 Motivation

When shopping for a trip, users often need to switch between weather forecasts, social media, and online shopping platforms. They have to check the weather, look for local outfit inspiration, search for similar products, and decide whether the items are suitable, compatible, and within budget. This makes the shopping process time-consuming and requires a lot of searching and decision-making. So we hope to build an intelligent shopping agent for the specific scenario of travelling, combining previous travel information retrieval, outfit inspiration discovery, product search and shopping decisions into one smooth shopping process. This would help reduce the time taken for customers to repeatedly search, compare, and filter products, while helping them quickly find complete and purchasable outfits that suit their specific travel needs.

1.3 Project Goal

Our goal is to develop a travel-oriented intelligent shopping agent that integrates contextual reasoning, multimodal fashion retrieval, real-product recommendation, and outfit compatibility assessment. The system aims to transform users' travel needs into practical shopping decisions and ultimately provide complete outfit recommendations that are travel-appropriate, fashionable, feasible under real-world constraints, and connected to real purchasable products when additional items are needed.

2 Project Background

2.1 Market Context

With the development of e-commerce and artificial intelligence, online fashion shopping is shifting from traditional keyword searching and product browsing toward more intelligent, personalised, and interactive experiences. Recent advances in conversational recommendation, multimodal models, and AI shopping agents allow systems to better understand user needs through natural language and images, then identify relevant products from a wide range of options.

When it comes to the area of fashion shopping, the item itself contains abundant visual and textual information such as colour, design, material, category, and style. Therefore, traditional text-based keywords alone may not fully capture users' fashion needs. Previous research has shown that multimodal representations combining images and text can better support fashion product understanding and recommendation. For example, FashionCLIP maps fashion images and natural language into a shared semantic space, enabling tasks such as fashion product retrieval and classification [1].

2.2 Challenges We Address

2.2.1 Fragmented Travel and Fashion Information

A major problem in travel fashion shopping is that the information needed to make outfit purchasing decisions is often scattered across different platforms and sources. Travel plans and personal needs come from users, weather information comes from weather services, destination fashion trends and outfit inspiration are mainly found on social media, while purchasable products are available on different e-commerce platforms. It is difficult to directly combine this scattered information and turn it into clear and actionable fashion requirements.

Therefore, the first problem addressed by the project is how to integrate travel-related information from different sources and reason about the fashion requirements for a specific trip.



Figure 2.1 Fragmented Travel and Fashion Information

2.2.2 Gap Between Fashion Inspiration and Purchasable Products

Another challenge in the current shopping process is the gap between fashion inspiration and real products. Users can find destination- or activity-related outfit inspiration on social media, but finding a suitable outfit image does not mean they can easily find visually and stylistically similar products that are actually available for purchase. On traditional e-commerce platforms, users often need to manually turn their outfit ideas into specific product search terms. Therefore, the second problem addressed by this project is how to connect destination-related fashion inspiration with real and purchasable products, reducing the gap between discovering outfit inspiration and finding actual products.

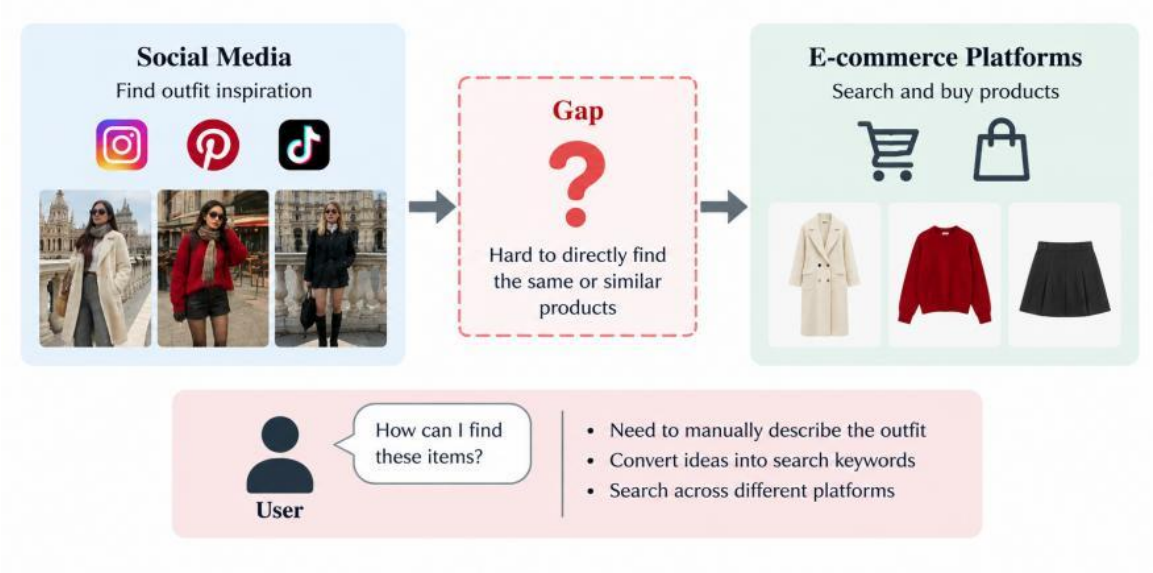


Figure 2.2 Gap Between Fashion Inspiration and Purchasable Products

2.2.3 From Single Products to Feasible Outfit Combinations

Finding relevant products does not necessarily lead to a suitable and complete outfit. Even when multiple products match the user's travel needs, users still need to decide which items work well together and whether the final combination meets practical constraints such as budget, product availability, and size. Therefore, the third problem addressed by this project is how to select compatible and feasible outfit combinations from multiple candidate products while satisfying real-world constraints.



Figure 2.3 Difficulty in Building Complete and Feasible Outfits

2.3 Market Landscape

In recent years, AI has been increasingly applied to online fashion shopping. Existing research and commercial products cover areas such as personalised recommendation, conversational shopping, multimodal product understanding, visual search, and intelligent outfit recommendation. Studies have shown that AI-driven fashion recommender systems use user preferences, visual product features, and other information to improve product discovery and provide more personalised shopping experiences [2].

In the commercial market, AI shopping tools are also shifting from traditional product search toward more conversational, personalised, and multimodal shopping experiences. Different products focus on areas such as digital wardrobe management, natural language product search, visual fashion discovery, and personalised recommendations. The following Business Case and Market Research chapter further examines the key market trends, representative competitors, user needs, and existing market opportunities in AI-powered fashion shopping, helping to clarify the positioning of our project in the travel fashion shopping scenario.

3 Business Case and Market Research

3.1 Business Context and Problem

Selecting outfits for a trip is a complex decision-making task. Users must consider the destination, travel dates, weather, planned activities, personal preferences and budget. However, the required information is often distributed across weather services, social media, e-commerce platforms and the user's own wardrobe.

Users therefore need to check the weather, search for outfit inspiration, find similar products and determine whether the selected items are suitable, compatible and within budget. This project addresses three related problems: fragmented travel and fashion information, the gap between fashion inspiration and purchasable products, and the difficulty of combining individual products into complete outfits that satisfy practical constraints.

WearTo addresses these problems through a travel-oriented intelligent shopping assistant. It converts natural-language travel requirements into contextual clothing needs and connects fashion inspiration, purchasable products and clothes already owned by the user to produce feasible outfit recommendations.

3.2 Market Trends and User Expectations

Online fashion shopping is moving from traditional keyword search towards more conversational, personalised and multimodal experiences. Related research has shown that, compared with traditional one-time recommendations, conversational recommender systems can better understand user needs and preferences through continuous interaction, allowing users to provide feedback and refine recommendations [3].

Fashion products also contain important visual information, such as colour, design, pattern and style. Multimodal models can combine text and images to support fashion understanding and retrieval [1].

Users expect recommendations that reduce decision effort while remaining relevant and transparent. Personalised recommender systems can use user preferences and behaviour to help users find more relevant products and improve their shopping decisions. The level of personalisation provided by recommendation agents can also influence users' decision quality and decision effort [4]. Clear product information and recommendation reasons can also reduce uncertainty in online shopping [5].

3.3 Competitive Landscape

Existing applications address different parts of the fashion shopping process.

Solution	Main focus	Position relative to WearTo
Acloset	Digital wardrobe management and personalised styling	Mainly focuses on clothes already owned by the user. WearTo also connects travel requirements to purchasable products.
Daydream	Conversational fashion shopping and product discovery	WearTo additionally considers travel context, weather, activities, practical constraints and outfit compatibility.
Pinterest Assistant	Visual fashion inspiration and shopping discovery	WearTo uses inspiration as one stage of a broader process that produces complete and feasible travel outfits.

Table 3.1 Comparison of Existing Fashion Applications and WearTo

These products demonstrate demand for conversational shopping, wardrobe support and visual fashion discovery. However, they generally focus on individual parts of the process rather than connecting travel requirements, inspiration, existing clothes and purchasable products within one recommendation workflow.

3.4 Market Opportunity and Business Value

WearTo considers the user's destination, dates, weather, activities, style preferences and budget before combining relevant products with clothes already owned by the user. The system recommends complete outfits rather than isolated products and considers contextual relevance, garment compatibility and practical constraints.

Each recommendation distinguishes owned items from suggested purchases and provides product information, purchase links and the cost of new items. Users can refine their requirements, request an explanation or confirm a final outfit.

The business value of the system lies in reducing the time and effort required to search for, compare and combine suitable products. It helps users move from an unstructured travel need to a smaller set of feasible and actionable outfit decisions.

4 System Design and Model

WearTo is designed as an interactive travel outfit recommendation system that connects conversational requirement understanding, contextual reasoning, multimodal retrieval, outfit compatibility prediction and constraint-based recommendation. The system is organised as a 14-stage pipeline across five modules: Understand, Inspire, Retrieve, Compose and Decide. This structure allows each module to perform a specialised reasoning task while exchanging validated data through defined interfaces.

4.1 Use Cases

The system supports four main user interactions, as shown in Figure 4.1.

UC01 - Submit a Trip Shopping Request. The user provides the destination, travel dates and planned activities together with relevant preferences and constraints, such as budget, currency, style, size, colour and excluded items. The system converts the natural-language request into structured information and asks focused clarification questions when required information is missing or ambiguous. The user reviews and confirms the interpreted request before external retrieval begins.

UC02 - View Outfit Recommendations. After the recommendation pipeline is completed, the user receives up to three feasible outfit recommendations. Each recommendation presents the relevant products, owned items, prices, availability information and purchase links. The user may also request an explanation of the recommendation, including its compatibility score, product relevance and constraint status.

UC03 - Confirm a Final Outfit. The user confirms one of the outfits displayed in the latest recommendation round by selecting its recommendation card. The system verifies that the selection belongs to the current result set, records the confirmed outfit and proceeds to the preview stage. A portrait-based preview is generated only with explicit user consent; otherwise, the system can provide a flat-lay preview or retain the product-card presentation.

UC04 - Refine Recommendations through Chat. The user can modify requirements such as budget, activities, style, exclusions or item preferences. The system represents the changes as a structured patch, preserves unchanged requirements and determines which processing stages are affected. Only those stages are rerun, after which a new Top-3 set is presented. The refinement loop can continue until the user confirms an outfit.

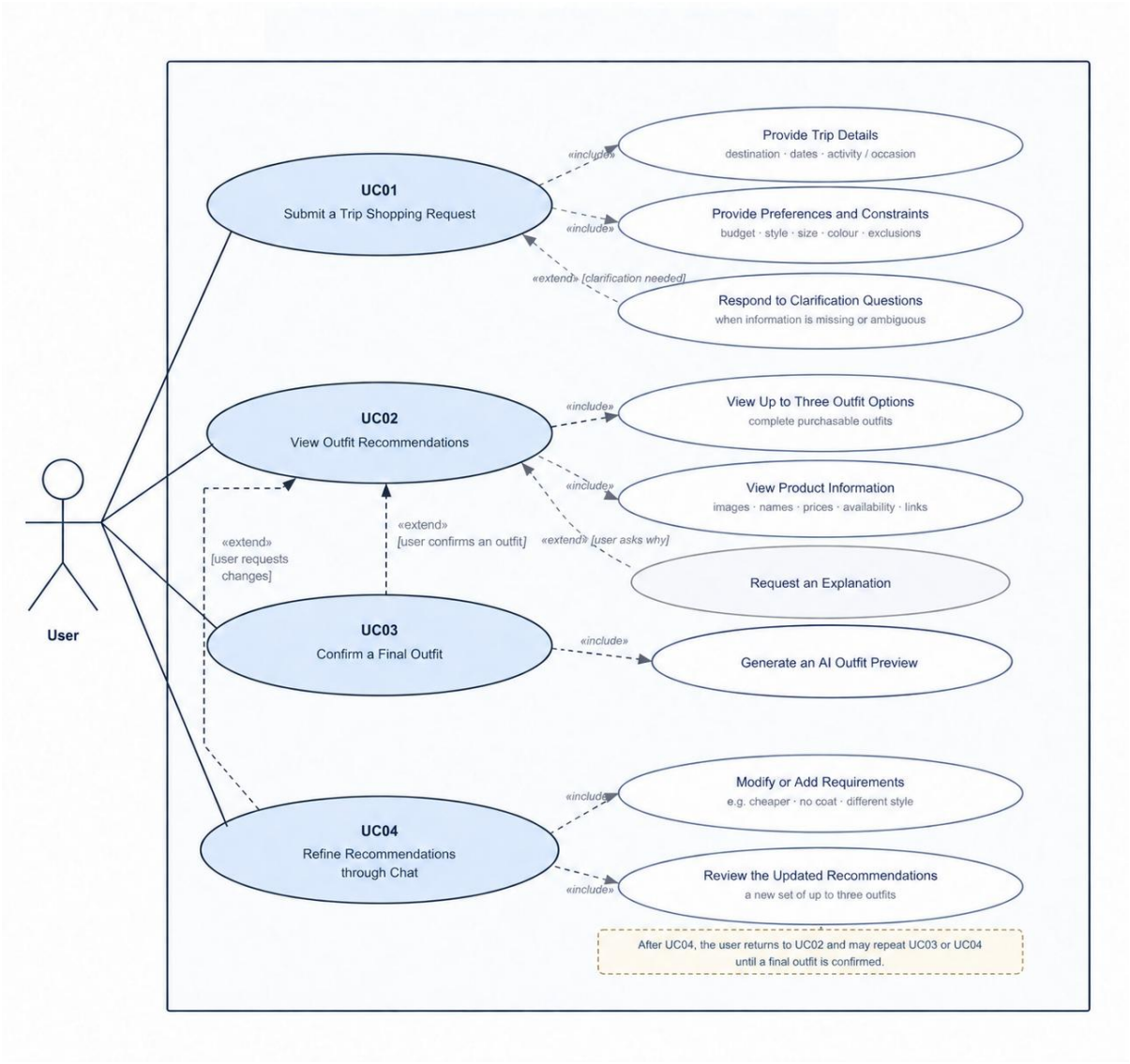


Figure 4.1 Use Case Diagram of the WearTo Intelligent Travel Style and Shopping Assistant

4.2 User Interaction Flow

Figure 4.2 illustrates the interaction between the user, the state-management layer and the recommendation pipeline.

The process begins when the user submits a travel outfit request. The interaction layer extracts and validates the requirements, checks them against the current dialogue state and requests clarification when mandatory information is incomplete or invalid. Confirmed requirements are stored before the recommendation pipeline is scheduled.

The recommendation pipeline enriches the request with weather, activity and place information, retrieves and reranks visual references and purchasable products, generates candidate outfits, calculates compatibility

scores and applies budget, availability and reliable-size constraints. The remaining outfits are ranked and the Top-3 results are returned to the interface.

After reviewing the recommendations, the user can request an explanation, modify the requirements or confirm an outfit. An explanation does not change the recommendation state. A modification updates the stored request and triggers selective re-execution of the affected stages, while confirmation records the selected outfit from the latest recommendation round. The confirmed outfit can then be sent to the preview service, with a product-card fallback if preview generation is unavailable.

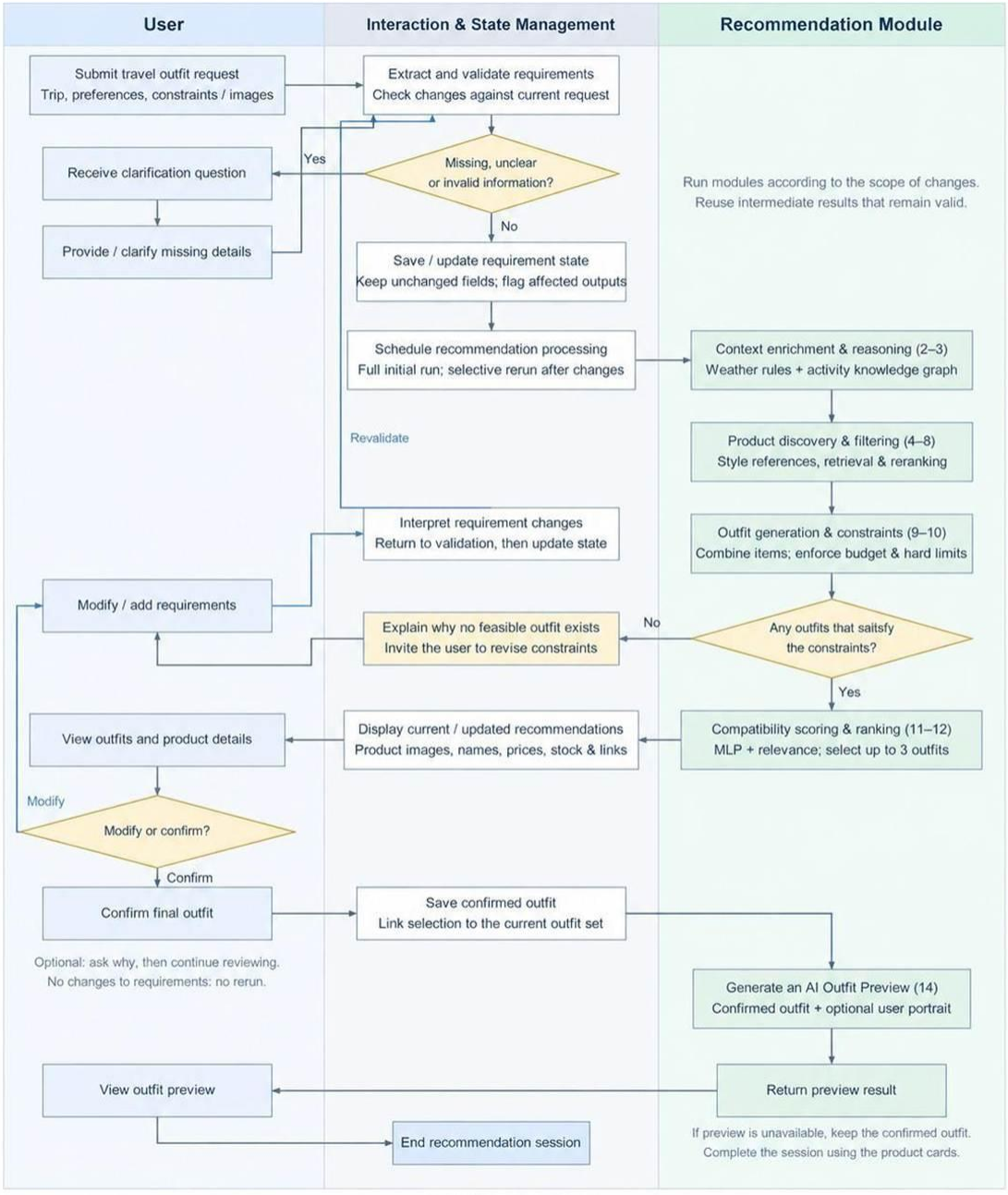
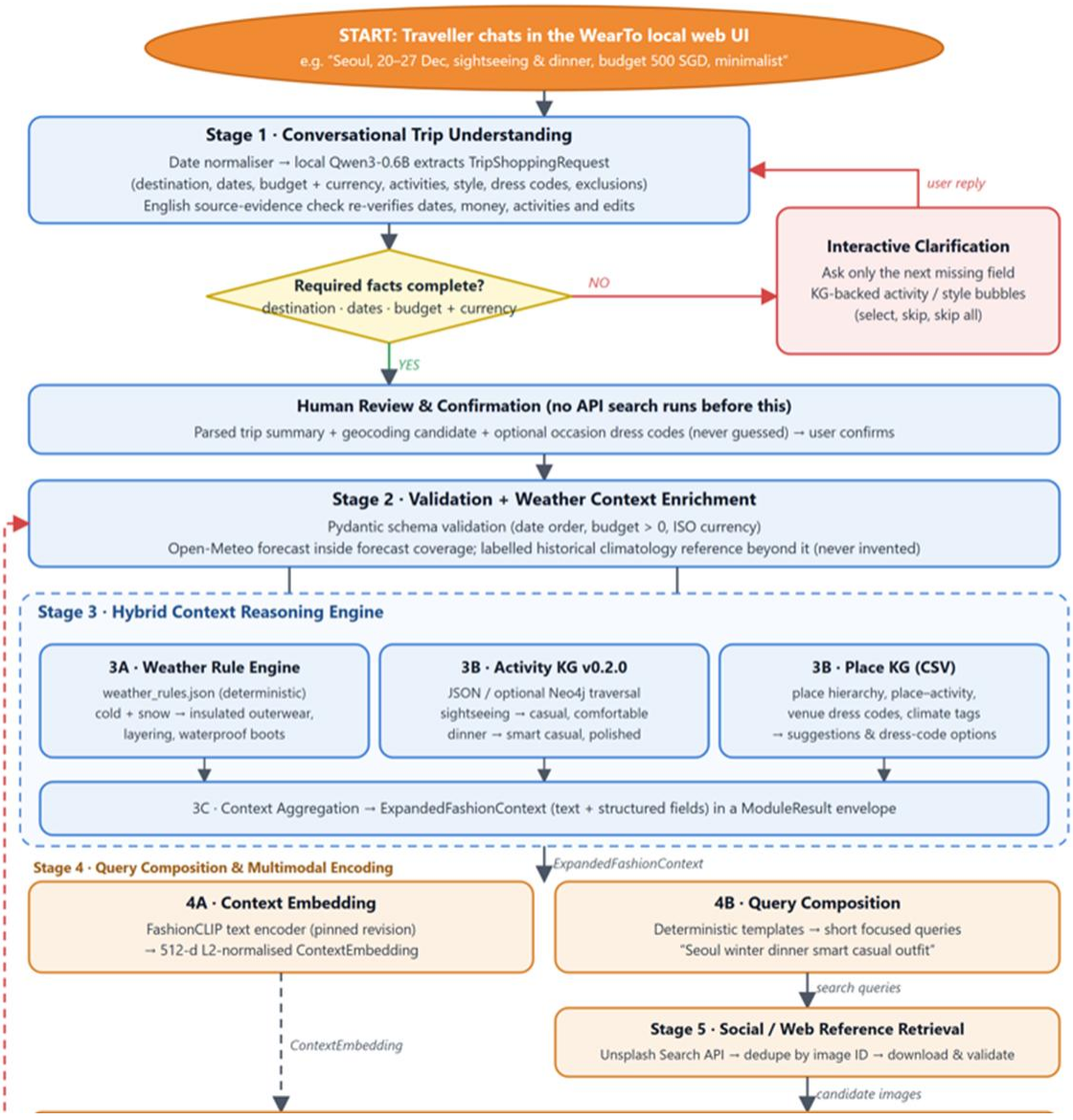


Figure 4.2 User Interaction Flow of the WearTo System

4.3 System Architecture Overview

The project architecture is implemented as a 14-stage hybrid reasoning pipeline organised into five modules: Understand (Stages 1–3), Inspire (Stages 4–6), Retrieve (Stages 7–8), Compose (Stages 9 and 11) and Decide (Stages 10, 12–14). It separates conversational intent extraction, deterministic weather reasoning, knowledge-graph activity and place reasoning, multimodal reference discovery, hybrid e-commerce retrieval, product reranking, learned outfit compatibility, hard-constraint filtering, final ranking, selective refinement and generative visualisation. Every module exposes service.run(payload) returning a common ModuleResult and is registered as one LangChain tool; the WearTo local web interface drives the pipeline. Large artefacts such as embeddings, images and outfit combinations move between modules as run-folder file handoffs rather than through the language model, and external providers (Open-Meteo, Unsplash, eBay, ECB, OpenAI) are replaceable adapters.



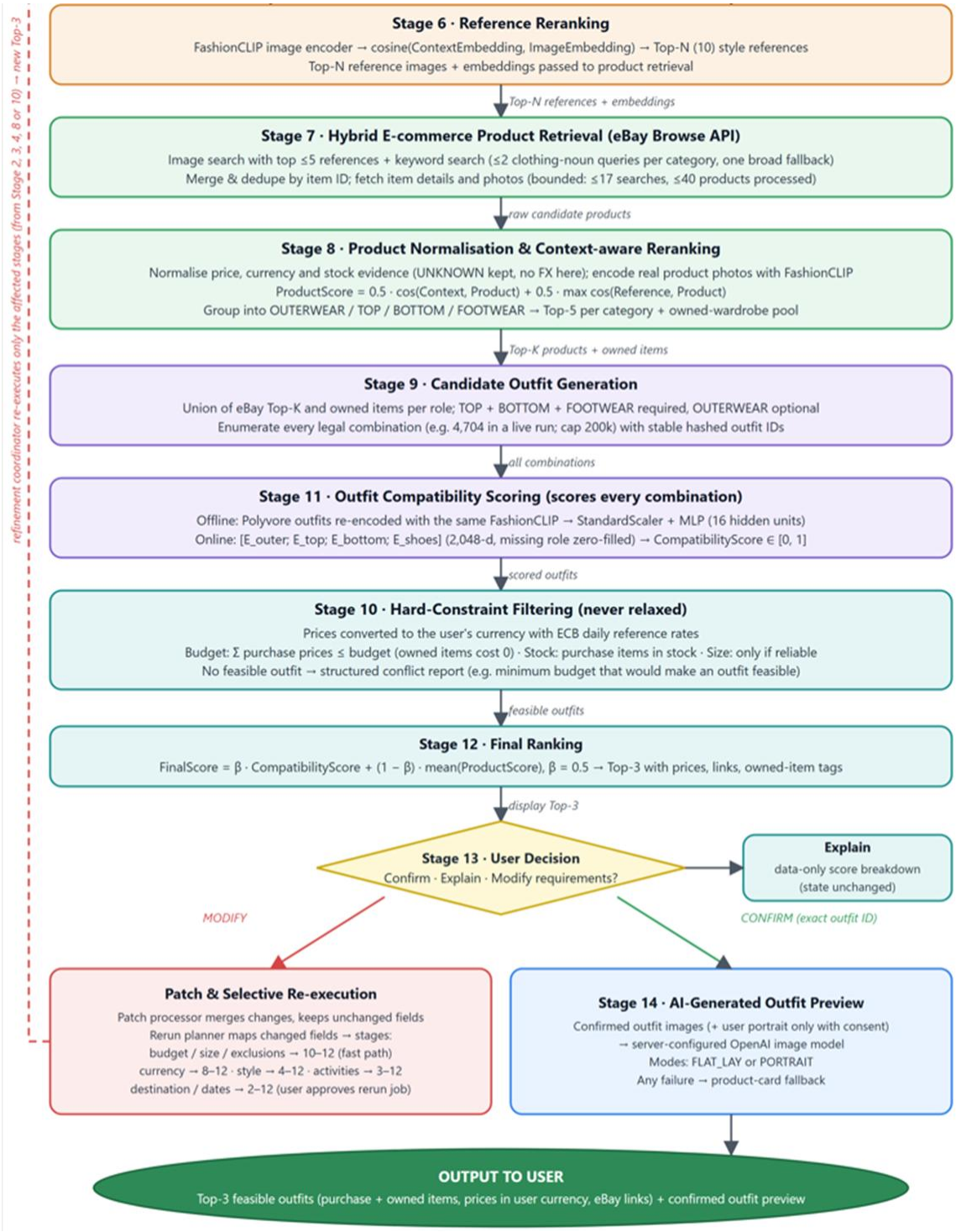


Figure 4.3 System Architecture

The 14-stage pipeline can be understood as four connected phases. Stages 1–3 turn the user's chat into a human-confirmed, validated trip request enriched with weather, activity and place requirements. Stages 4–8 use this context to discover and rerank visual style references and to retrieve and rank purchasable eBay products alongside the user's own wardrobe. Stages 9–12 combine these items into outfits, score every

combination for compatibility, enforce budget, stock and size constraints, and rank the feasible outfits. In the implementation, compatibility scoring (Stage 11) runs on all combinations before hard-constraint filtering (Stage 10), so the scored outfits can be re-filtered after a budget or size change without rescoring. Stages 13–14 support confirmation, explanation and selective refinement, followed by an AI-generated preview of the confirmed outfit.

The following table maps the functional workflow to the corresponding processing stages, showing how each function is supported by the system architecture. It also provides a reference for the detailed implementation in Chapter 5.

Functional Workflow Step	Corresponding System Design Implementation
User chat → LLM → structured request	Stage 1 (M1): Dates are normalised, then local Qwen3-0.6B extracts a TripShoppingRequest (destination, dates, budget and currency, activities, style, dress codes, exclusions). An English source-evidence layer re-checks dates, money, activities and edits. The conversation controller asks only for the next missing required field and offers KG-backed activity/style choices (select, skip or skip all).
Human review → validated request + weather	Stage 1 → 2 (M1): The user reviews the parsed trip, geocoding result and optional dress codes; no external search runs before confirmation. Pydantic validates the schema, and Open-Meteo supplies a forecast inside forecast coverage or a labelled historical climatology reference beyond it.
Weather rules + Knowledge Graphs → fashion context	Stage 3 (M1): The deterministic weather rule engine produces Weather-Based Clothing Requirements; the Activity KG (JSON or optional Neo4j) produces Activity-Based Style Requirements and alternative aesthetics; the Place KG provides place–activity links and venue dress codes. Context aggregation produces the ExpandedFashionContext.
Expanded context → embedding + query composition	Stage 4 (M2): The pinned FashionCLIP text encoder produces a 512-d L2-normalised ContextEmbedding, and deterministic templates generate short, focused search queries.
Search queries → social reference images	Stage 5 (M2): Queries are sent to the Unsplash Search API; results are deduplicated by image ID, downloaded and validated.
Reference images → similarity reranking + handoff	Stage 6 (M2): FashionCLIP image embeddings are compared with the ContextEmbedding by cosine similarity and the Top-N (10) references are kept. Embeddings, context and model metadata are handed to product retrieval as files, with a model-identity check.
Reference images + queries → eBay products	Stage 7 (M3): Hybrid eBay Browse API retrieval: image search with up to five top references plus keyword search (up to two clothing-noun queries per category, one broad fallback). Results are merged and deduplicated by item ID within a bounded search budget.
Products → context-aware reranking → Top-K per category	Stage 8 (M3): Price, currency and stock evidence are normalised (UNKNOWN is kept). Real product photos are encoded with FashionCLIP; ProductScore = 0.5 × context similarity + 0.5 × maximum reference similarity. Products are grouped

Functional Workflow Step	Corresponding System Design Implementation
	into OUTERWEAR, TOP, BOTTOM and FOOTWEAR, keeping the Top-5 per category plus the user's owned-wardrobe pool.
Top-K products + wardrobe → candidate outfits → compatibility	Stages 9 and 11 (M4): Every legal combination of eBay and owned items is enumerated (TOP, BOTTOM and FOOTWEAR required, OUTERWEAR optional) with stable outfit IDs. Each is scored by the Polyvore-trained compatibility MLP over the concatenated 2,048-d FashionCLIP vector, giving a CompatibilityScore.
Scored outfits → constraints → Top-3 recommendation	Stages 10 and 12 (M5): Prices are converted to the user's currency with ECB reference rates. Hard constraints (budget over purchase items only, stock, and size when reliable) are never relaxed; if no outfit is feasible, a conflict report is returned. Feasible outfits are ranked by FinalOutfitScore = β × CompatibilityScore + (1 − β) × OutfitProductRelevance, where OutfitProductRelevance is the mean ProductScore (β = 0.5), and the Top-3 are displayed.
User confirms, asks why, or modifies requirements	Stage 13 (M5): Confirm saves the exact outfit ID as the final selection. Explain returns a data-only score breakdown. Modify creates a patch that preserves unchanged requirements; the rerun planner maps changed fields to stages (budget, size, exclusions: 10–12; currency: 8–12; style: 4–12; activities: 3–12; destination, dates: 2–12). After user approval, a deterministic coordinator re-executes only those stages and shows a new Top-3. This repeats until the user confirms.
Confirmed outfit (+ portrait) → preview	Stage 14 (M5): The confirmed outfit's product images, plus a user portrait only with explicit consent, are sent to a server-configured OpenAI image model in FLAT_LAY or PORTRAIT mode. Any failure falls back to product cards.

Table 4.1 Functional workflow steps mapped to the system design

4.4 Design Summary

The final system integrates conversational understanding, contextual reasoning, multimodal retrieval, compatibility prediction and constraint-based ranking within a stateful recommendation workflow. Learned models provide semantic and compatibility signals, while deterministic validation and hard constraints ensure that the final recommendations remain feasible and traceable. The system also allows the user to inspect, modify and confirm its outputs instead of treating recommendation as a one-way prediction.

5 System Development & Implementation

The system is divided into five main modules, each developed primarily by a different team member. For clarity, the terms Student 1 to Student 5 used throughout this report refer to the members responsible for the corresponding modules. Specifically, Student 1 refers to the member responsible for Section 5.1, Student 2 for Section 5.2, and so on through Student 5 for Section 5.5. These labels are used consistently to indicate module ownership and individual development responsibilities. Module N (MN in Chapter 4) corresponds to Student N.

5.1 User Requirement Understanding and Contextual Reasoning

The project implemented a travel requirement understanding and contextual reasoning module that converts natural-language requests into structured, editable and traceable clothing requirements. The module combines a pretrained language model, deterministic rules and knowledge graphs to provide a consistent input for downstream inspiration retrieval, product search and outfit recommendation.

5.1.1 Requirement Understanding and Dialogue Management

A locally deployed Qwen3-0.6B model extracts the destination, travel dates, budget, currency, activities and style preferences. Model outputs are checked using Pydantic schemas, business rules and evidence from the user's original message. User-provided facts are stored separately from inferred requirements to reduce the risk of introducing unsupported preferences or contextual information. Missing, ambiguous or invalid information triggers a focused clarification while preserving details already collected.

The dialogue manager distinguishes proposed information from confirmed requirements. Users can provide details incrementally and revise existing requirements through set, add and remove operations. Unspecified fields are retained, and invalid updates do not overwrite the previous state. Price preferences are also distinguished from budget limits: a request for a cheaper option does not create an arbitrary new budget.

Date handling supports standard numeric dates, slash-separated dates, abbreviated English months, Chinese date expressions and selected relative expressions. Users can supply a missing year without repeating the entire request. Ambiguous dates require clarification rather than silent interpretation. Activities and styles are optional; the system can offer knowledge-graph-supported choices, but suggestions become request facts only after explicit selection. These interactions were integrated into the team's web interface through multiple-selection controls, selection feedback and trip confirmation.

5.1.2 Weather Enrichment and Knowledge-Based Reasoning

The weather component integrates Open-Meteo geocoding and forecasts with deterministic rules that produce clothing categories and functional requirements, including insulation, layering and rain protection. For trips outside or crossing the forecast window, the system uses explicitly labelled historical climate references rather than presenting historical statistics as trip-date forecasts.

The historical dataset contains 180 monthly summaries for 15 cities across 12 countries, derived from ERA5 daily data for 2015–2024. A shareable JSON snapshot and a local SQLite database retain location, provenance and statistical-method information. Country-level inputs require city clarification, and unsupported cities have an explicit data acquisition path. Unavailable data remains unknown rather than being replaced by assumed clear weather or a neighbouring city's conditions.

The Activity Knowledge Graph contains 16 activity types and 22 Style nodes, with 90 nodes and 203 relationships overall. Bounded one-hop and multi-hop traversal connects activities, occasions, formality, styles and attributes while retaining conditions and supporting evidence. Alternative aesthetics are represented separately from functional clothing needs. Activities such as dining require additional dress-code context where appropriate, rather than implying a universal level of formality.

A complementary Place Knowledge Graph covers 100 places across eight countries and represents venue requirements, activity choices and conditions such as entering an interior space. Its 753 base records are exported as 601 nodes and 1,214 relationships, including aliases and provenance structures. Outputs from both graphs are combined with weather requirements and user preferences to produce structured context and an ExpandedFashionContext representation for downstream retrieval.

5.1.3 Knowledge Review and Traceability

Knowledge maintenance distinguishes source-supported statements from project heuristics and preserves applicability conditions, source reading levels and review status. The project owner completed the review of the existing knowledge, and approval was recorded for the activity nodes, relationships and sources, together with all 753 Place Knowledge Graph records.

The review process retains record-level snapshots, reviewer identity, timestamps, content fingerprints and specification fingerprints. Validation precedes the application of review decisions, and backups support recovery. Dependency checks prevent approval of a source or place from automatically approving every associated assertion. Changes to reviewed content invalidate the corresponding approval and require further review. Runtime snapshots and export files are kept consistent with the reviewed data.

The system supports both local Python reasoning and a Neo4j backend for the Activity Knowledge Graph. Python performs conditional reasoning, while Neo4j provides storage and bounded-path verification. Live queries confirmed agreement between the two backends for the evaluated scenarios. The Place Knowledge Graph currently uses local reasoning.

5.2 Inspiration Module

The Inspiration module (Stages 4–6) turns Module 1's fashion context into a ranked set of real-world style reference images, plus their FashionCLIP embeddings. It hands both to Module 3 as files. The code lives in

`SystemCode/clips/travel_agent/inspiration/` and runs as a standalone pipeline, a service and a LangChain tool (`search_references`).

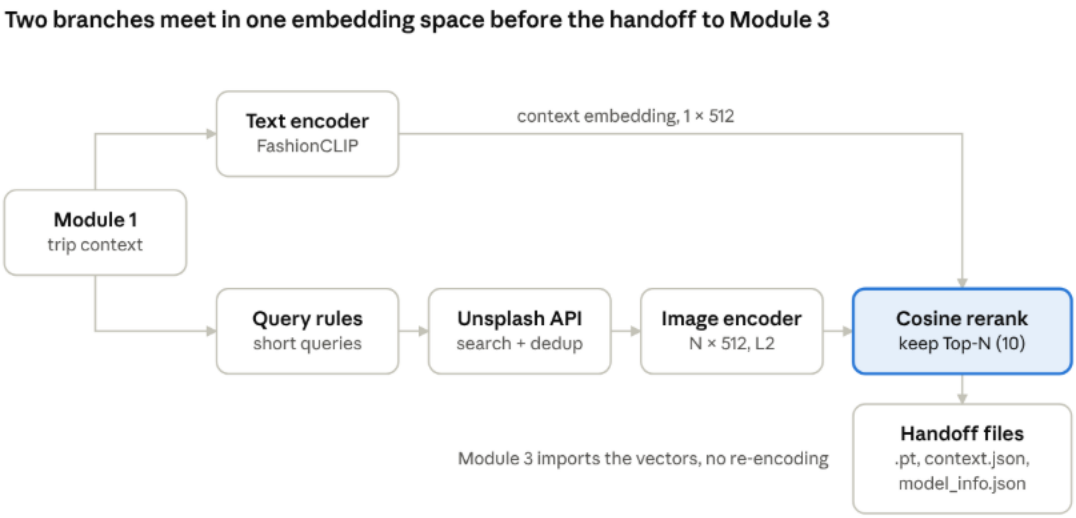


Figure 5.1 M2 Inspiration pipeline · two branches, one ranking step, file handoff

The context is embedded once on the top branch; the bottom branch finds and embeds candidate photos; cosine similarity joins them, and only the Top-N leave the module.

5.2.1 Inputs from Module 1

The service reads Module 1's `ModuleResult.data` and proceeds only when `ready_for_downstream` is true. It splits the data into one string to embed and structured fields to build queries from:

Input	Source field in Module 1 output	Used for
Expanded fashion context (one string)	`expanded_fashion_context`	FashionCLIP text embedding
Destination	`trip_request.destination`	Search queries
Season	`season`	Search queries
Weather clothing requirements	`weather_clothing_requirements`	Search queries (outerwear, top/bottom/footwear)
Activity style requirements	`activity_style_requirements`	Search queries (one per activity)
User style	`trip_request.style`	Search queries

Table 5.1 Inputs from Module 1

5.2.2 Processing Pipeline

`run_inspiration_pipeline()` runs these steps (FashionCLIP `patrickjohncyh/fashion-clip` at a pinned revision, 512-d embeddings, L2-normalised):

- **Load and encode the context.** FashionCLIP's text encoder embeds the expanded context once into a 1 × 512 ContextEmbedding (text truncated at the 77-token limit).

- **Generate search queries.** Deterministic templates build short queries: one for the style, one per activity, one per outerwear garment and one per top × bottom × footwear combination, e.g. “Seoul winter dinner smart casual polished outfit”.
- **Retrieve and deduplicate.** Each query calls the Unsplash Search API; failed queries are logged and skipped. Results are deduplicated by image ID, keeping the original rank for evaluation.
- **Load and encode images.** Images are downloaded, EXIF-corrected and converted to RGB; failed downloads are dropped. The image encoder embeds them in batches of 8 into an N × 512 matrix.
- **Rank and select.** As both sides are unit vectors, cosine similarity is a dot product. Images are sorted by score and the Top-N (default 10) are kept, aligned row for row with their embeddings.

The pipeline returns the context embedding, the Top-N records and embeddings, and evaluation data (every candidate's original and FashionCLIP rank) used in the Experiments section.

```
SocialScore(j) = cos(e_context, e_image_j) = e_context · e_image_j
```

5.2.3 Downstream Product Retrieval Integration

`save_inspiration_handoff()` writes each run to its own folder with four files:

File	Contents
`phase_2_output.pt`	Context embedding, Top-N records, Top-N image embeddings
`expanded_fashion_context.txt`	The context string that was embedded
`context.json`	The structured context used for queries
`model_info.json`	Model ID, revision, preprocessing, version, dimension and normalisation for both the context and image embeddings

Table 5.2 Handoff files

Module 3 compares `model_info.json` with its own model identity and, on a match, reuses these vectors instead of recomputing them, after checking dimensions, norms and row order.

5.2.4 Service and Agent Integration

`inspiration.service.run(payload)` wraps the pipeline in the shared `ModuleResult` contract, and `tools.py` exposes it to the LangChain agent as `search_references`; the live web pipeline calls the same functions. It returns `needs_input` for unusable Module 1 output, `no_candidates` when no image can be retrieved or loaded, `error` for unexpected failures and `ok` with the Top-N references and handoff paths on success.

5.2.5 Design Decisions

- **Separate text for embedding and for search.** The long context is embedded once; search uses short templated queries, because one long query mixes conflicting intents (casual sightseeing vs polished dinner).

- **One shared, pinned embedding space.** Modules 2 and 3 use the same FashionCLIP checkpoint and normalisation, so context, references and products are directly comparable.
- **Files instead of chat for tensors.** Embeddings move as `.pt` files with model metadata, keeping the agent context small and the vectors exact.
- **Fail soft per item, strict per contract.** One failed query or image is skipped; an unusable overall result becomes a clear status, never an empty success.

5.2.6 Testing

Service, tool and Module 3 handoff behaviour is covered by mocked pytest tests that run in CI without network access, API keys or model downloads. Live Unsplash retrieval, FashionCLIP embedding compatibility, cosine ranking and handoff row alignment were checked with separate manual scripts.

5.3 Product Retrieval, Wardrobe Image Understanding and System Integration

This component focused primarily on product retrieval and existing wardrobe image understanding, alongside the integration of other team members' modules into the WearTo website to connect the dynamic Student 1–5 pipeline.

5.3.1 Product Acquisition, Cleaning and Standardisation

Shared product data formats and module interfaces were established to support both a prepared catalogue and real eBay products. Using Python, Pandas and Pydantic, data cleaning, deduplication, category mapping and standardisation of colours, sizes, prices, currencies and availability were implemented, while preserving source information and processing records.

Image search and keyword search were integrated through the eBay Browse API. Candidates from both routes are merged, deduplicated and ranked together, subject to request budgets, processing limits and supplementary category queries. After Sandbox testing, an account-deletion notification service was implemented and deployed, the callback and local cleanup mechanism were verified, and Production access was successfully enabled.

Availability is recorded from explicit API estimates; missing or contradictory evidence remains UNKNOWN. A product listing is not treated as proof of stock. Original prices and currencies are retained, with currency conversion delegated to Student 5.

5.3.2 FashionCLIP Encoding and Category-Specific Top-K

Frozen FashionCLIP was used to generate 512-dimensional image and text embeddings, with consistent model identity, preprocessing and normalisation. Embeddings are cached to reduce repeated inference.

Product relevance combines travel-context similarity with the strongest match to the selected social reference images:

ProductScore = alpha × ContextSimilarity + (1 − alpha) × MaximumSocialReferenceSimilarity (alpha = 0.5)

The system returns separate Top-5 pools for tops, bottoms, footwear and outerwear. Category shortages are reported without duplicate padding. An independent adapter was also developed for the team's PT, TXT and JSON inputs, checking tensor shapes, image–embedding ordering and model identity. When compatibility cannot be established, reference images can be downloaded and re-encoded locally.

5.3.3 Wardrobe Understanding and Downstream Handoff

The wardrobe module supports single-garment photo uploads, preprocessing, category/subcategory/colour/style prediction, FashionCLIP encoding and structured storage. Uncertain attributes remain UNKNOWN.

Owned clothes do not compete in the eBay Top-K ranking. They form a separate wardrobe pool that joins the product pools during Student 4 outfit construction. Timestamped outputs preserve products, wardrobe items, images, embeddings, scores and provenance. The product handoff package uses relative image paths for transfer between team members.

The 21 approved baseline wardrobe images are included in GitHub. Future private uploads, profiles, credentials and runtime data remain local.

5.3.4 Dynamic Integration and the WearTo Website

Building on the original Student 4 model, dynamic input adaptation was implemented to replace fixed-result-file dependencies, and training and runtime embedding processing were aligned. Outfit generation combines retrieved products and owned clothes, providing complete combinations and compatibility scores downstream.

The travel Chatbox, social inspiration carousel, profile and wardrobe-upload pages were developed, and Student 5 was connected to the current run's output. Integration issues were resolved, including new trips being misrouted as recommendation refinements, destination images not updating, owned clothes being incorrectly labelled as retail stock, and empty product descriptions blocking preview requests.

5.4 Outfit Compatibility Prediction and Ranking

5.4.1 Data Preprocessing

The outfit compatibility module was developed using the Polyvore dataset, which provides outfit combinations with binary compatibility labels. Each outfit was represented by a list of product IDs and a label indicating whether the combination was compatible (1) or incompatible (0).

To align the dataset with our system requirements, accessories such as earrings, rings, sunglasses, etc. were removed, and five original garment categories were mapped into four standardised categories: OUTERWEAR, TOP, BOTTOM (including pants and skirts), and FOOTWEAR. Product IDs were matched with their corresponding images in the dataset, and products without valid image matches were removed from each outfit. The remaining products were then sorted according to the predefined category order. Outfits containing fewer than two valid products or multiple products belonging to the same standardised category were excluded. The original compatibility labels were preserved for all retained outfits. After preprocessing, the dataset was reduced from 7,076 to 2,377 outfits, retaining approximately 33.6% of the original records.

The preprocessing stage produced a cleaned outfit dataset, a product-category mapping file, and a collection of selected product images for subsequent feature extraction.

5.4.2 FashionCLIP Embedding Extraction

FashionCLIP was used to extract visual features from product images. Each unique product was encoded into a 512-dimensional image embedding, representing its visual characteristics in a numerical feature space.

The extracted embeddings were stored in an NPZ file to avoid repeated image encoding during model training.

For each outfit, product embeddings were concatenated in a fixed category order: OUTERWEAR, TOP, BOTTOM, and FOOTWEAR. Missing categories were represented by zero vectors, producing a consistent 2048-dimensional outfit representation.

5.4.3 Compatibility Model Training

A Multilayer Perceptron (MLP) classifier was implemented to predict outfit compatibility.

The dataset was divided into training (70%), validation (15%), and testing (15%) subsets using stratified sampling. A StandardScaler was applied to normalise the input features before training.

The MLP consisted of one hidden layer with 16 neurons and ReLU activation. L2 regularisation (alpha = 0.1) and early stopping were applied during training.

The model was evaluated using classification accuracy, precision, recall, and F1-score. The trained StandardScaler and MLP classifier were saved together as a scikit-learn Pipeline to ensure consistent preprocessing during inference.

The final model achieved a test accuracy of approximately 61.9%, indicating moderate predictive performance on the processed Polyvore dataset.

5.4.4 Outfit Generation, Scoring and Ranking

A Python compatibility service was developed to integrate the trained model with the upstream product recommendation results.

The service reads Top-K candidate products provided by the upstream module and generates outfit combinations under three constraints: TOP, BOTTOM, and FOOTWEAR are mandatory; OUTERWEAR is optional; and each category can contain at most one product.

With top 5 candidate products per category, the service generates 750 valid outfit combinations.

Each outfit is converted into a 2048-dimensional embedding and passed to the trained model. The predicted probability of the compatible class is used as the outfit's compatibility_score.

All generated outfits are ranked in descending order of compatibility score. The service returns a JSON-formatted string containing rank-based outfit IDs, compatibility scores, and complete product information.

This module enables the system to evaluate complete outfit combinations rather than ranking individual products independently, providing an additional compatibility-based ranking stage for the overall recommendation pipeline.

5.5 Constraint Optimisation, Final Recommendation and User Interaction

The final decision layer of WearTo (Student 5) converts upstream candidate outfits and compatibility scores into feasible, ranked and interactive recommendations. Its implementation covers hard constraints, final scoring and Top-3 selection, recommendation explanations, requirement refinement, confirmation, preview integration, and testing of both the Student 5 module and the complete five-module pipeline.

5.5.1 Hard-Constraint Filtering and Data Validation

The module accepts a structured trip request and the candidate outfits scored by Student 4. Pydantic contracts are used to validate required fields, score ranges, product information and unique outfit identifiers. Deterministic constraints are implemented for budget, availability and reliable size evidence. Purchase totals exclude user-owned clothes, and size filtering is applied only when the user supplies a size and the product has reliable size data.

Hard constraints are never relaxed automatically. Rejected outfits retain structured reasons, and a no-feasible result returns a conflict summary rather than an unsuitable recommendation. Prices and budgets are also normalised to one currency before comparison; invalid currency evidence is rejected.

5.5.2 Final Scoring and Top-3 Recommendation

For each feasible outfit, OutfitProductRelevance is calculated as the mean of the item-level ProductScore values supplied by the product-retrieval stage. This captures how strongly the products in the outfit relate to the travel context and selected fashion references. The final score combines this relevance with the CompatibilityScore supplied by Student 4:

OutfitProductRelevance(o)
= mean(ProductScore(item) for item in outfit o)

FinalOutfitScore(o)
= β × CompatibilityScore(o)
+ (1 − β) × OutfitProductRelevance(o)

beta is configurable between 0 and 1. Lower values emphasise contextual product relevance, while higher values emphasise garment compatibility; beta = 0.5 is the balanced default. The module sorts feasible outfits by final score, applies deterministic tie-breaking and returns a Top-3 RecommendationSet containing stable internal identifiers, products, purchase totals and score components.

A "Why this outfit?" function is implemented for the Top-3. It explains the selected recommendation using its position, compatibility, product relevance, final score, purchases, cost, budget status and existing wardrobe items. The explanation is derived from stored evidence rather than generated claims, and selections from an earlier recommendation round are rejected.

5.5.3 Requirement Refinement, Confirmation and Selective Re-execution

After viewing the Top-3, the user can request an explanation, confirm an outfit or modify the requirements. Changes use explicit set, add and remove operations, preserving untouched fields and rejecting invalid updates. A valid change invalidates the previous recommendation set and triggers only the affected pipeline stages: for example, a budget change starts from constraint filtering, whereas destination or date changes require the contextual and retrieval stages to run again.

In the web interface, the user confirms an outfit by clicking the confirmation control on the selected recommendation card; the user does not need to enter an identifier. Internally, the interface submits that card's stable outfit_id, allowing the backend to verify that the selection belongs to the latest recommendation round. Unknown identifiers, repeated confirmation and selections from an earlier round are rejected. A successful confirmation stores an immutable snapshot of the selected recommendation for the preview stage.

5.5.4 AI Outfit Preview and Web Integration

A confirmed recommendation is connected to the preview pipeline. The system supports a flat-lay preview and, when the user has supplied an authorised portrait, a portrait-based preview. Preview prompts are built from the confirmed outfit and the available product and portrait images.

The preview layer includes a mock provider for testing, an OpenAI Images adapter and a product-card fallback. Together, the web workflow presents Top-3 recommendations, explanations, refinement, confirmation and preview in one interaction.

5.5.5 Testing Architecture

The first testing layer consists of component and service tests under SystemCode/clips/tests/recommendation/, covering Student 5 constraints, ranking, state transitions, explanation and preview. The second layer uses a reusable five-module harness with an entry-point registry,

payload builders, hand-off validators and stage/run records. Both the integration tests and command-line runner reuse this harness, allowing them to exercise the public service.run interfaces without hard-coding another member's output and to record the exact module and reason when a pipeline stops.

5.5.6 System Integration

The implementation also includes the Student 4-to-Student 5 adapter and hand-off checks across the five public module entry points. The integration layer calls the other modules through their defined interfaces and validates their outputs without reimplementing their internal logic. Incomplete runs retain their actual stopping stage and reason rather than being recorded as successful.

6 Experiments

6.1 Requirement Understanding and Contextual Reasoning

6.1.1 Objective

To assess whether Module 1 converts natural-language trip requests into correct structured requirements, maintains the dialogue state correctly across follow-up turns, and produces the expected weather and knowledge-graph reasoning results. The evaluation compares the raw Qwen3-0.6B output with the complete rule-augmented system, which adds source-evidence checks, schema validation and dialogue-state management.

6.1.2 Evaluation Datasets

Evaluation was organised into complementary layers covering language understanding, dialogue state, knowledge reasoning and weather processing. Test cases included incomplete information, ambiguity, negation, invalid inputs and successive requirement changes, alongside complete requests.

Evaluation set	Design and coverage
Multilingual requirement understanding set	32 scenarios comprising 40 turns: 25 English, 13 Chinese and two mixed-language turns. Covers extraction, missing information, currency ambiguity, cross-year dates, negation, exclusions, invalid inputs and three continuous dialogues.
English development subset	25 English turns selected from the multilingual set, used for development and regression evaluation of extraction, updates, explanation requests and confirmation.
Supplementary English diagnostic set	16 additional turns covering new destinations and styles, monetary precision, missing currencies, date changes, exclusion removal and ambiguous confirmation.
Dialogue controller scenario set	Six scenarios comprising 19 turns, covering date formats, year-only clarification, ambiguous-date protection, incremental information collection, activity selection and optional-field skipping.
Place reasoning scenario set	16 scenarios with predefined expectations for seasonal differences, indoor and outdoor settings, entry conditions, venue requirements and missing information.

Table 6.1 Module 1 evaluation sets

Raw model outputs and outputs after deterministic processing were scored separately. Full correctness required agreement on the expected facts, empty fields, intent, validation behaviour and state transitions, so that many unchanged fields could not mask a missed update or an unsupported addition. Safe rejection of invalid inputs was recorded separately from successful extraction. Single-turn cases supplied the expected

previous state to isolate extraction errors, whereas continuous dialogues carried forward the actual generated state, allowing errors to accumulate.

6.1.3 Requirement Understanding Results

Early evaluation identified omitted facts, invented preferences, incorrect handling of negation and full-request responses where a local update was required. Development therefore focused on prompt refinement, source-evidence checks, vocabulary normalisation, patch-based updates and explicit confirmation.

Dataset	Initial raw model	Initial rule-augmented system	Improved raw model	Improved rule-augmented system
Multilingual understanding set	3/40	14/40	4/40	23/40
English development subset	4/25	21/25	4/25	22/25
Supplementary English diagnostic set	1/16	15/16	1/16	16/16

Table 6.2 Full correctness before and after development

Full correctness on the multilingual set increased from 35.0% to 57.5%, a gain of 22.5 percentage points. The English development subset reached 88.0% and the supplementary English diagnostic set reached 100%. The raw model remained at 4/40, 4/25 and 1/16, so the improvement comes from the deterministic evidence checks and state management rather than from model training or fine-tuning.

The number of continuous dialogues reaching the expected final state increased from zero of three to two of three; both English dialogues succeeded, while the Chinese dialogue did not. Follow-up intent classification and intended changed-field correctness were both 8/8 in the English development subset, and 7/7 and 5/5 respectively in the supplementary English set. Neither English evaluation recorded changes to fields the user had not asked to modify. All six dialogue controller scenarios (19 turns) met their predefined expectations.

6.1.4 Knowledge Reasoning and Weather Validation

Validation	Result	Interpretation
Activity Knowledge Graph tests	14/14 passed	Covers scenarios, paths, conditions, review filtering and export consistency.
Place Knowledge Graph tests	60/60 passed	Includes review persistence, dependency checks, stale approvals, re-review, conflict detection, rollback and recovery.
Place reasoning scenarios	16/16 passed	Checks required outputs, prohibited inferences and missing-information responses with draft knowledge disabled.
Neo4j and local reasoning comparison	4/4 matched	Hiking, museum visits and dining under different dress-code and traversal-depth settings; retained paths passed live Cypher verification.
Strict review-chain validation	Complete chains for 12 requirements across three scenarios	Checks the dependencies supporting each result rather than an isolated approval flag.
Weather processing checks	2/2 passed	Verifies a live city forecast, a future-trip historical lookup and the distinction between the two data types.

Table 6.3 Knowledge reasoning and weather validation

6.1.5 Discussion

Structured generation alone was insufficient for reliable requirement understanding: raw-model performance remained far below the rule-augmented system. Output formatting also did not improve consistently. Strict JSON objects in the multilingual set decreased from 30/40 to 8/40, although a parser accepting a single code fence could still parse all 40 outputs. Because the prompt changed from a general to an English-focused configuration, the overall gain should not be attributed to a single component.

The English improvements did not transfer to Chinese: none of the 13 Chinese turns met the strict full-correctness criterion, so English remains the primary supported workflow. Safe rejection and extraction success were also kept distinct. In the English development subset, the system correctly rejected an impossible date, a negative budget and unsupported currency precision, giving 25/25 for validation outcomes but 22/25 for full correctness.

These results demonstrate correctness within the development scenarios rather than held-out generalisation. Knowledge approval was recorded by a single reviewer without a measured inter-reviewer agreement study, and field-level F1, a controlled ablation and independent quality ratings remain future work (Section 8.1).

6.2 Inspiration Module

This section reports the offline evaluation of the Inspiration module's FashionCLIP reranking. The experiment, labels and metrics are in SystemCode/clips/travel_agent/inspiration/notebooks/04_retrieval_evaluation.ipynb.

6.2.1 Objective

To test whether reranking Unsplash search results by FashionCLIP cosine similarity to the user's expanded fashion context places more relevant style references at the top than the search engine's own order.

6.2.2 Experimental Setup

Item	Setting
Trip context	Seoul, winter; sightseeing and dinner; minimalist style
Context requirements	Winter coat or puffer, sweater/knitwear, long trousers, closed shoes or boots; warm, insulated, layerable; sightseeing casual, dinner smart casual
Queries	9 template-generated queries sent to the Unsplash Search API
Candidate pool	43 retrieved records → 19 unique images after deduplication by image ID
Baseline	Original Unsplash retrieval order (aggregated across queries)
Proposed method	FashionCLIP (pinned patrickjohncyh/fashion-clip) cosine reranking, Top-10 kept

Table 6.4 Experimental setup

6.2.3 Relevance Labelling

All 19 candidates were labelled manually on a graded 0–2 scale:

- 2 – Highly relevant: winter-appropriate clothing, suitable for sightseeing or dinner, broadly minimalist.
- 1 – Partially relevant: some suitable garments, but important weather or activity requirements are missing.
- 0 – Irrelevant: beachwear, summer-only outfits or unrelated images.

This gave 9 highly relevant, 7 partially relevant and 3 irrelevant images. As an independent cross-check, Gemini 2.5 Flash was used as a VLM judge with the same rubric and a structured JSON output (label, reason, confidence).

6.2.4 Metrics

- Precision@K: share of the Top-K images labelled highly relevant (strict: only label 2 counts).
- NDCG@K: graded ranking quality with gain 2^label − 1 and a log2 position discount, normalised by the ideal ordering of all 19 labels.

- Inter-rater agreement: exact agreement and quadratic-weighted Cohen's kappa between manual and Gemini labels.
- Retrieval diagnostics: duplicate rate, unique yield per query and candidate relevance rate.

6.2.5 Results

Metric	Original order	FashionCLIP	Improvement
Precision@5	0.40	0.60	+0.20
Precision@10	0.40	0.60	+0.20
NDCG@5	0.6012	0.6864	+0.0853
NDCG@10	0.6029	0.7091	+0.1061

Table 6.5 Ranking quality: original Unsplash order vs FashionCLIP reranking (manual labels)

FashionCLIP reranking improved every metric. It placed 3 highly relevant images in the Top-5 and 6 in the Top-10, against 2 and 4 in the original order. The NDCG gains (+0.09 at K = 5, +0.11 at K = 10) show that the most relevant images also move earlier in the list.

Measure	Value
Exact agreement (manual vs Gemini)	42.1% (8 of 19)
Quadratic-weighted Cohen's kappa	0.53 (moderate)
Label distribution: manual	9 highly · 7 partially · 3 irrelevant
Label distribution: Gemini	5 highly · 8 partially · 6 irrelevant

Table 6.6 Agreement between manual and VLM-judge labels

Gemini was consistently stricter than the human annotator, and all 11 disagreements were only one grade apart, which is why the weighted kappa is higher than exact agreement. The reported ranking metrics use the manual labels.

Diagnostic	Value
Records retrieved / unique images	43 / 19
Duplicate rate	55.8% (24 records)
Queries adding no new images	Queries 5–9
Strict relevance rate (label 2)	47.4%
Relaxed relevance rate (label ≥ 1)	84.2%

Table 6.7 Retrieval diagnostics

Queries 1–3 contributed 18 of the 19 unique images; later queries mostly returned photos already seen. 84.2% of the unique candidates were at least partially relevant.

6.3 Product Retrieval, Wardrobe Understanding and System Integration

6.3.1 Objective

To verify that Module 3 retrieves real purchasable products from eBay Production, builds category-specific Top-K pools alongside the user's own wardrobe, and connects all five modules into a working end-to-end recommendation workflow.

6.3.2 Functional Validation

Validation	Result
Live product retrieval	eBay Production returned real products; hybrid image-and-keyword acquisition and category-specific Top-K selection were verified.
Complete web workflow	Multi-turn input and trip confirmation connected to live weather, Unsplash, eBay, compatibility scoring and final filtering, producing three recommendations.
Recorded end-to-end run	358 candidate products retrieved; 80 products encoded with FashionCLIP; five products per category and 21 owned items retained; 4,704 outfit combinations scored; three recommendations returned by Student 5.
AI preview	In a separate live test, confirming a recommendation produced a real OpenAI flat-lay preview displayed in the browser.
Automated regression suite	857 tests passed and 7 were skipped in the full local environment; dependency-lock and formatting checks passed.

Table 6.8 Product retrieval and integration validation

In the recorded run, 13 of the 21 owned items had a supported category and joined the 20 retrieved Top-K products, giving category pools of 6 outerwear, 7 tops, 12 bottoms and 8 footwear items. The remaining eight owned items stayed UNKNOWN and were kept out of outfit-role assignment. Enumerating every combination with optional outerwear gives (6 + 1) × 7 × 12 × 8 = 4,704 outfits, compared with 750 when only the five retrieved products per category are used (Section 5.4.4).

6.3.3 Retrieval Evaluation Design

An offline evaluator was developed to compare three ranking strategies on saved Production retrieval runs without making new API calls: context similarity only (alpha = 1), reference similarity only (alpha = 0) and the deployed hybrid (alpha = 0.5). It reuses each run's indexed product images and FashionCLIP vectors and stops unless the hybrid Top-K reproduces the original output exactly. A pilot covers three trip scenarios (Seoul, Tokyo and Beijing) and records the candidate funnel, category coverage, field completeness and Top-K overlap between the strategies.

For relevance, the evaluator produces a shuffled, blinded labelling sheet that hides the strategy, score and rank. Two reviewers grade every candidate from 0 (irrelevant) to 3 (highly relevant) and assign its true category; the

adjudicated labels then give Precision@5, Recall@5 within the indexed pool, NDCG@5, mean relevance and category accuracy and macro-F1. The human labelling had not been completed for this report, so no relevance metrics are reported for Module 3.

6.3.4 Discussion

These results demonstrate functional integration rather than recommendation quality. Product relevance, wardrobe classification accuracy, user satisfaction and the choice of alpha still require independent labels and systematic comparison. Availability reflects the API snapshot at retrieval time and is not a checkout guarantee, and a single recorded run does not guarantee three recommendations for every request.

6.4 Outfit Compatibility Model Evaluation

This experiment evaluates the performance of the MLP-based outfit compatibility model using the preprocessed Polyvore dataset. After preprocessing, 2,377 outfits with binary compatibility labels were retained. Each product was represented by a 512-dimensional FashionCLIP image embedding, and outfit embeddings were constructed by concatenating four garment categories in a fixed order, with zero padding for missing categories, resulting in 2048-dimensional feature vectors.

The dataset was divided into training (70%), validation (15%), and testing (15%) sets using stratified sampling. A StandardScaler and an MLPClassifier with one hidden layer of 16 neurons, ReLU activation, L2 regularisation (alpha = 0.1), and early stopping were used for compatibility prediction. Model performance was evaluated using accuracy, precision, recall, and F1-score.

The model achieved a validation accuracy of **57.42%** and a test accuracy of **61.90%**. On the test set, the compatible class achieved a precision of **0.64**, recall of **0.68**, and F1-score of **0.65**.

These results suggest that the model can learn some compatibility-related patterns from FashionCLIP embeddings, although its classification performance remains limited. Since outfit compatibility is subjective and depends on relationships between garments, the predicted compatibility probabilities are used primarily for relative outfit ranking rather than absolute compatibility judgments.

6.5 Final Outfit Ranking and Beta Selection

This evaluation measures how the weighting parameter beta changes the final Top-3 outfit recommendations. The two-stage experiment selects beta = 0.50 because it achieves the strongest coarse-grained preference result and the best ranking quality among nearby settings.

6.5.1 Objective

The final ranking combines outfit-level compatibility from Student 4 and mean product relevance from Student 3. The experiment determines whether changing beta materially affects the Top-3 ranking and identifies a balanced final setting.

$$\mathbf{FinalOutfitScore = \beta \times CompatibilityScore + (1 - \beta) \times OutfitProductRelevance}$$

6.5.2 Experimental Setup

Item	Setting
Input	Real Student 4 output ranked_outfits.json
Candidate outfits	750
Feasible after hard constraints	725
Rejected outfits	25 budget conflicts
Destination and dates	Seoul, 20 to 27 December 2026
Activities and style	Sightseeing, dinner and minimalist style
Test budget	USD 500
Coarse beta values	0.00, 0.25, 0.50, 0.75 and 1.00
Fine beta values	0.40, 0.45, 0.50, 0.55 and 0.60
Changed variable	Beta only
Output	Top-3 ranked outfits per beta

Table 6.9 Controlled experimental configuration

The production Student 5 recommendation service was rerun for every beta value. The request, candidates, currency conversion and hard constraints remained unchanged, so ranking differences were caused by beta rather than different inputs.

6.5.3 Manual Evaluation

Unique outfits were shuffled into blind review sheets. Beta values, ranks and system scores were hidden during labelling. A consistent single-reviewer simulated evaluation assessed all outfits, and repeated outfit IDs received the same labels in both stages.

Score	Outfit compatibility	Product relevance	Overall preference
0	Clear colour, style or seasonal conflict	Does not satisfy the request	Would not recommend
1	Wearable with a noticeable inconsistency	Partially satisfies the request	Acceptable but not preferred
2	Coherent colour, style and seasonal use	Strong match for the request	Suitable for direct recommendation

Table 6.10 Manual scoring rubric

The assessment considered Seoul winter weather, sightseeing, dinner and minimalist styling. Budget was not scored manually because it had already been enforced as a hard constraint.

6.5.4 Evaluation Metrics

- Mean overall preference: mean 0 to 2 preference grade of the three recommended outfits.
- NDCG at 3: whether more highly preferred outfits appear earlier in the Top-3 ranking.
- Top-3 composition: candidate replacement and rank changes as beta varies.
- Ranking stability: similarity of recommendations under neighbouring beta values.

Mean overall preference was the primary criterion. NDCG at 3 was the tie-breaker, followed by proximity to the balanced midpoint beta = 0.50.

6.5.5 Coarse-Grained Experiment

Beta	Rank 1	Rank 2	Rank 3	Mean preference	NDCG at 3
0.00	O0561	O0011	O0355	1.33	0.821
0.25	O0011	O0561	O0183	1.33	1.000
0.50	**O0011**	**O0027**	**O0240**	**1.67**	**1.000**
0.75	O0007	O0013	O0044	1.33	1.000
1.00	O0001	O0020	O0035	1.33	1.000

Table 6.11 Coarse beta results

The coarse sweep produced 12 unique outfits across 15 ranking positions, confirming that beta materially changes the recommendation set. Beta = 0.50 achieved the highest mean preference of 1.67. All other settings achieved 1.33, indicating that heavier reliance on either scoring signal reduced the perceived balance of the final recommendations.

6.5.6 Fine-Grained Experiment

A second experiment examined the neighbourhood around beta = 0.50.

Beta	Rank 1	Rank 2	Rank 3	Mean preference	NDCG at 3
0.40	O0011	O0240	O0027	1.67	0.951
0.45	O0011	O0240	O0027	1.67	0.951
0.50	**O0011**	**O0027**	**O0240**	**1.67**	**1.000**
0.55	O0011	O0027	O0029	1.33	1.000
0.60	O0011	O0013	O0033	1.33	1.000

Table 6.12 Fine beta results

Beta values from 0.40 to 0.50 retained the same three outfits, demonstrating local stability. Beta = 0.50 placed both highly preferred outfits, O0011 and O0027, in the first two positions and therefore achieved NDCG at 3 of 1.000. At beta = 0.55 and beta = 0.60, lower-rated alternatives entered the Top-3 and reduced mean preference to 1.33.

Based on the coarse- and fine-grained experiments, beta = 0.50 was selected as the final setting because it achieved the best balance between mean preference and ranking quality.

7 Findings and Discussion

The project demonstrates the feasibility of combining language understanding, knowledge-based reasoning, semantic retrieval, compatibility prediction and deterministic decision rules within a travel-oriented outfit recommendation system. In Module 1, deterministic evidence checks and dialogue-state management improved full correctness on the multilingual evaluation set (40 dialogue turns across 32 scenarios) from 35.0% to 57.5%, while all six dialogue-controller scenarios achieved their expected outcomes. However, these improvements did not transfer reliably to Chinese, and English therefore remains the primary supported workflow.

The retrieval modules also demonstrated the value of combining external search with a shared semantic representation. FashionCLIP reranking increased both Precision@5 and Precision@10 from 0.40 to 0.60, moving more relevant inspiration images towards the top of the results. Product retrieval then combined image and keyword search, standardised the returned product information and produced category-specific Top-K candidates. In one complete recorded run, the system retrieved 358 candidate products, encoded 80 products, generated 4,704 outfit combinations (including the user's owned wardrobe items) and returned three final recommendations.

The compatibility model achieved a test accuracy of approximately 61.9% on the processed Polyvore dataset. This moderate result, together with uncertainty introduced during dataset preprocessing, indicates that compatibility scores should be treated as relative ranking signals rather than absolute judgements of outfit quality. To support final decision-making, Module 5 combined compatibility with product relevance while applying budget, availability and reliable-size requirements as non-relaxable constraints. The beta comparison selected $\beta = 0.50$ as the most balanced setting. The resulting recommendations can also be explained, confirmed or refined, with unchanged requirements preserved and only the affected processing stages rerun.

Overall, the findings demonstrate the feasibility of the proposed architecture and show that the five modules can operate together as a complete recommendation workflow. The evaluation results provide initial support for the system's retrieval, ranking and refinement functions. Further evaluation across broader travel scenarios and with additional human evaluators would help assess the robustness and generalisability of the recommendations.

8 Future Work

The current prototype shows that the five modules can operate together as a complete recommendation workflow, but several components were validated only on development scenarios or small evaluation sets. Future work therefore focuses on five areas, combining the directions set out in the project proposal with the limitations identified by each module during implementation and evaluation.

8.1 Systematic Evaluation of the End-to-End System

The most important next step is to evaluate the complete system rather than individual components. Final recommendations should be rated for travel relevance, outfit compatibility and constraint satisfaction by multiple independent evaluators, with reviewer agreement reported, across a broader range of destinations, activities and budgets.

Module-level evaluation should also be strengthened. Requirement understanding requires field-level F1, held-out test data and a controlled ablation study to separate the contributions of prompts, rules and knowledge graphs. The Inspiration module could compute Precision@K and NDCG@K automatically on each live run using its stored evaluation data, and product retrieval and wardrobe classification require independent labelled evaluation sets so that scoring weights and retrieval strategies can be compared systematically.

8.2 Contextual Reasoning, Multilingual Support and Personalisation

Module 1 performs reliably on the covered English scenarios, but its improvements did not transfer to Chinese. Future work will address the remaining errors in currency omission, activity normalisation, update intent and dialogue-state evolution so that multilingual dialogues become a supported workflow.

The knowledge base can be extended with further weather rules, activity types and destination and cultural knowledge. Weather reasoning can incorporate attributes such as humidity, independently validated thresholds and special environments such as mountain or indoor conditions. Knowledge currently approved by the project owner should also be reviewed by independent or domain-expert reviewers. In addition, richer user preference modelling would allow recommendations to adapt more closely to individual style and previous choices.

8.3 Fashion Inspiration and Product Retrieval

The Inspiration module currently scores whole Unsplash scenes, so an image can rank highly because of its setting rather than its clothing. Cropping or segmenting garments before embedding would make similarity scores reflect the clothing more directly. Inspiration and product sources can also be expanded to additional social media and e-commerce platforms, taking recency, engagement and larger product catalogues into account.

On the product side, a FashionCLIP zero-shot category fallback would reduce the number of live products left in the UNKNOWN category, and variant-level stock and size evidence would make availability and size filtering more reliable.

8.4 Outfit Compatibility Modelling

The compatibility model was trained on a reduced Polyvore dataset, and its row-level split allowed garments to appear in both the training and test sets. Future work will re-evaluate the model using item-disjoint partitions and reviewed labels, and will measure how closely its rankings correspond to human preferences. Larger or travel-specific training data could also improve the low absolute scores observed on real products. Outfit generation should additionally apply a weather-driven outerwear policy rather than generating combinations both with and without outerwear.

8.5 Recommendation, Interaction and System Efficiency

In the decision layer, the dependency map used for selective re-execution must be kept aligned with upstream interfaces as the modules evolve. AI outfit previews currently support decision-making but do not guarantee real-world fit, fabric behaviour, colour accuracy or availability, so linking previews more closely to product images and size information is a natural extension.

System efficiency can be improved by caching the FashionCLIP model, reusing handoffs across refinements that leave the travel context unchanged and limiting overlapping queries to conserve API quota. Sustained notification processing and cross-device handoffs would further support deployment beyond a local prototype.

Appendix A: Project Proposal

This appendix summarises the project proposal submitted on 13 September 2026. It records the project as originally proposed, before implementation; Chapters 4 and 5 describe the system as finally built. The proposal's business case and market research are already reflected in Chapter 3 and are not repeated here. Section A.8 reproduces the data contract from the proposal's own appendix.

A.1 Project Details

Item	Details
Project title	Intelligent Travel Style & Shopping Assistant
Course	NUS-ISS Intelligent Reasoning Systems (practice module)
Team	Group 17
Date of proposal	13 September 2026
Team members	A0353874R Agarwal Hritik Kumar; A0353592X Jiang Yushan; A0351286B Lu Yujing; A0314662H Ma Bofei; A0330891A Zhang Yurun

Table A.1 Project details

A.2 Background and Objectives

Choosing suitable outfits for a trip is a multi-constraint decision. Clothing depends on the destination, travel dates, weather, planned activities, personal style and budget, and the information needed is scattered across weather services, social media, online shops and the user's own wardrobe. Travellers therefore switch repeatedly between sources to find inspiration, search for similar products and check whether items are suitable, compatible and affordable.

The proposal set out to build a travel-oriented intelligent shopping agent that understands requirements expressed in natural language, reasons over weather and activity knowledge, retrieves fashion inspiration and real products, and combines them with clothes the user already owns. The goal was to recommend complete outfits that are travel-appropriate, fashionable, feasible under real-world constraints and linked to purchasable products.

A.3 Proposed Scope

- **Input:** destination, travel dates, planned activities, budget and optional style preferences in natural language, with optional photos of existing clothes; the system asks for clarification when essential details are missing.
- **Reasoning:** weather data with rule-based reasoning, and an activity knowledge graph, derive the clothing and style requirements for the trip.
- **Retrieval:** outfit inspiration from social platforms or prepared reference datasets, and candidate products from an e-commerce source or prepared catalogue.
- **Outfit generation:** products and uploaded clothes are grouped by category and combined into outfits, filtered by budget, availability and reliable size data, and ranked by contextual relevance and compatibility. Only new purchases count towards the budget.
- **Output:** the three highest-ranked outfits, separating owned clothes from suggested purchases, with images, product details, purchase links and the cost of new items, plus an optional visual preview.
- **Out of scope:** comprehensive wardrobe management, automated purchasing and guarantees of physical fit.

A.4 Proposed Reasoning Techniques

Technique	Proposed use
Rule-based reasoning	Map temperature, rainfall and snowfall to clothing categories and functional requirements such as insulation, layering and weather resistance, keeping decisions traceable.
Knowledge-graph reasoning	An activity knowledge graph of activities, occasions, formality levels, styles and attributes; single- and multi-hop paths turn planned activities into style requirements.
Constraint filtering and recommendation	Enforce hard constraints (budget for new purchases, availability, size where reliable), then rank feasible outfits by contextual relevance and compatibility to produce three recommendations.
Multimodal reasoning	FashionCLIP places the travel context, reference images, products and uploaded clothes in one embedding space; an MLP trained on Polyvore outfits scores outfit-level compatibility.

Table A.2 Reasoning techniques

A.5 Data and Resources

Resource	Proposed sources
Fashion and activity knowledge	Curated occasion, formality and attire guidance (for example the Emily Post Institute), recording source, applicable context and review status.
Social and web references	Candidates assessed: Instagram Graph API, Pinterest API v5, Flickr API, Reddit Data API, TikTok Research API and the Unsplash API as a practical fallback.
E-commerce products	Candidates assessed: eBay Browse API (image search), Lazada Open Platform, ZALORA Seller Center, Amazon Product Advertising API, Shopify Storefront and Google Merchant, with a prepared catalogue as a fallback.
Outfit compatibility data	Curated Polyvore outfit dataset for training the compatibility model.
Runtime context	Weather forecasts, with labelled historical or seasonal data beyond the forecast range, and the user's requests.

Table A.3 Data sources considered

At the proposal stage a demo used Qwen2.5-1.5B-Instruct to extract destination, activities, style, occasion and preferences into JSON, then retrieved three products from a mock dataset by TF-IDF cosine similarity.

A.6 Evaluation Plan

Component	Baseline / comparison	Suggested metrics
Trip extraction	Prompted Qwen vs Qwen + LoRA (if implemented)	Field accuracy / exact match, valid JSON rate, F1 for extracted fields
Weather reasoning	Rule engine vs manually defined expected outputs	Rule coverage, constraint correctness
Activity KG	Direct one-hop lookup vs multi-hop reasoning	Expert/human agreement, coverage, path validity
Social reranking	Original search order vs FashionCLIP reranking	Precision@K, NDCG@K, human relevance rating
Product retrieval	Reference-only vs context-only vs hybrid ProductScore	Precision@K, Recall@K, human relevance rating
Category normalisation	E-commerce metadata vs FashionCLIP zero-shot	Accuracy, macro-F1, confusion matrix
Compatibility model	Heuristic / cosine baseline vs trained MLP	ROC-AUC, F1 / accuracy, ranking metric if supported

Component	Baseline / comparison	Suggested metrics
Final ranking	Different alpha / beta values	Human relevance rating, constraint satisfaction rate, NDCG@3
Multi-turn refinement	Full-request replacement vs patch-based state update	Intent accuracy, patch field F1, joint goal accuracy, unchanged-field preservation, rounds to confirmation

Table A.4 Evaluation plan proposed

A.7 Anticipated Challenges

Challenge	Planned mitigation
External data and API availability (access limits, rate limits, instability)	Cached or historical weather data, prepared social and product datasets as backups, and replaceable data interfaces.
Incomplete or inconsistent product information	Standardise product data across sources; use FashionCLIP zero-shot classification when categories are missing; state limits instead of filtering unreliably.
No outfit satisfies all constraints	Never relax hard constraints automatically; explain which constraints conflict and let the user adjust them.
Ambiguous feedback during conversation	Keep a structured dialogue state, update only clearly changed requirements and ask a short clarification question when intent is unclear.

Table A.5 Challenges and mitigations in the proposal

A.8 Data Contract

The proposal defined the main data objects exchanged between modules. A TripShoppingRequest holds only facts supplied by the user; missing optional values stay empty, and the language model must not invent weather, style or activity details.

```
{ "destination": "Seoul", "start_date": "2026-12-20", "end_date": "2026-12-27",
  "activities": ["sightseeing", "dinner"], "budget": 500, "currency": "SGD", "style": [] }
```

After validation, the retrieved WeatherContext is added to form the EnrichedTripContext. The weather rule engine and the activity knowledge graph then produce two separate reasoning outputs:

```
{ "trip_request": { ... },
  "weather_context": { "temp_min_celsius": -4, "temp_max_celsius": 5, "snow": true,
                       "precipitation_probability": 0.35, "weather_class": "COLD" } }

{ "weather_clothing_requirements": {
    "outerwear": ["winter coat", "puffer jacket"], "top": ["sweater", "knitwear"],
    "bottom": ["long trousers"], "footwear": ["closed shoes", "boots"],
    "properties": ["warm", "insulated", "layerable", "weather-resistant"] },
  "activity_style_requirements": {
    "sightseeing": ["casual", "comfortable"], "dinner": ["smart casual", "polished"] } }
```

A session-level state keeps the latest confirmed requirements, the recommendation round, the outfits last shown and the confirmation status. A message such as “Make it cheaper and do not include a long coat” becomes a request patch rather than a full replacement; fields absent from the patch keep their values:

```
{ "intent": "MODIFY_REQUIREMENTS",
  "set": { "budget": 400 },
  "add": { "excluded_items": ["long coat"] },
  "remove": {} }
```

Appendix B: Course Mapping

B.1 User Requirement Understanding and Contextual Reasoning

System Functionality	Relevant Course	Knowledge / Techniques Applied
Natural Language Requirement Understanding	MR	Natural language understanding using a pretrained language model (Qwen3-0.6B) to extract structured travel requirements from user requests.
Dialogue Management and Requirement Refinement	CGS	Context-aware dialogue management, clarification of ambiguous information, and maintenance of user requirements across multiple interactions.
Weather-Based Clothing Reasoning	RS	Rule-based reasoning using weather conditions to derive clothing requirements such as insulation, layering, and rain protection.
Activity and Place Knowledge Graph Reasoning	RS	Knowledge representation and graph-based reasoning using entities, relationships, conditions, and bounded graph traversal to infer contextual clothing requirements.
Knowledge Validation and Traceability	RS	Knowledge-base validation, provenance tracking, consistency checking, and maintenance of evidence-supported reasoning.

B.2 Inspiration Module

The Inspiration module draws mainly on Cognitive Systems (multimodal embeddings and semantic search), with Reasoning Systems for ranking and evaluation and Machine Reasoning for its rule-based query construction.

M2 functionality	Course	Knowledge / technique applied	How it is applied
FashionCLIP text and image encoders	CGS	Vision-language models, joint embedding space	Context text and outfit photos are projected into one 512-d space so they can be compared directly
Image preprocessing	CGS	Image processing for vision models	EXIF orientation correction, RGB conversion, CLIP pixel preprocessing, batched inference
Text tokenisation and truncation	CGS	NLP text representation	CLIP tokenizer with the 77-token limit for the expanded context
Semantic image search and reranking	CGS	Cognitive / semantic search	Keyword search candidates are reranked by meaning (cosine similarity), not by keyword match

M2 functionality	Course	Knowledge / technique applied	How it is applied
Cosine similarity Top-N ranking	RS	Similarity-based (content-based) recommendation	Candidates scored against the user context; Top-N kept as references for product recommendation
Hybrid pipeline	RS	Hybrid reasoning system	Deterministic query rules feed a learned embedding model; each part does what it is best at
Ranking evaluation with Precision@K and NDCG@K	RS	Evaluation of ranking and recommender systems	Top-5 and Top-10 rankings scored against manual relevance labels in an offline notebook
Rule-based query generation from KG/rule outputs	MR	Rule-based reasoning; using inferred knowledge	Slot-filling templates turn Module 1's weather-rule and Activity-KG requirements (plus destination, season and style) into focused search queries and context text
Readiness checks and status contract	MR	Explicit, explainable control logic	`ready_for_downstream`, `needs_input`, `no_candidates` and `error` decide whether and how the module proceeds

B.3 Product Retrieval, Wardrobe Image Understanding and System Integration

System Functionality	Relevant Course	Knowledge / Techniques Applied
Multimodal Product Retrieval and Ranking	MR	FashionCLIP-based image and text embeddings, cross-modal similarity matching, and weighted relevance scoring using travel context and social reference images.
Wardrobe Image Understanding	MR	Image-based garment classification and feature extraction to identify clothing categories, colours, styles, and visual representations of user-owned garments.
Product Data Standardisation and Filtering	RS	Rule-based validation and constraint handling for product categories, sizes, prices, availability, and missing or uncertain information.
Category-Specific Top-K Recommendation	RS	Constraint-based candidate selection and ranking, generating separate Top-K product pools for each clothing category while handling insufficient candidates.
Dynamic System Integration and User Interaction	CGS	Integration of multiple intelligent components into an interactive recommendation workflow, supporting travel requests, visual inspiration, wardrobe management, and dynamic recommendation updates.

B.4 Outfit Compatibility Prediction and Ranking

System Functionality	Relevant Course	Knowledge / Techniques Applied
FashionCLIP Image Embedding Extraction	MR	Multimodal representation learning; converting visual information into numerical embeddings for downstream reasoning.
Outfit Compatibility Prediction	MR	Feature representation and machine learning; using an MLP classifier to estimate compatibility between garments.
Constraint-Based Outfit Generation	RS	Rule-based reasoning; enforcing mandatory garment categories, optional outerwear, and category uniqueness constraints.
Outfit Compatibility Scoring and Ranking	RS	Combining learned compatibility predictions with rule-constrained candidate generation to rank valid outfit combinations.

B.5 Constraint Optimisation, Final Recommendation and User Interaction

Implemented functionality	Course mapping	Application of course knowledge and techniques
Budget, availability and reliable-size constraints	MR / RS	Encodes explicit domain and business rules, evaluates every candidate against them and returns traceable reasons when a rule is violated.
Structured conflict explanation	MR	Converts failed rule conditions into an interpretable reasoning result without silently relaxing the user's constraints.
Compatibility and relevance aggregation	RS	Combines two recommendation signals through a configurable weighted scoring function for multi-criteria decision-making.
Deterministic Top-3 ranking	RS	Scores, orders and selects feasible alternatives while applying reproducible tie-breaking rules.
Beta sensitivity experiment	RS	Evaluates how the weighting parameter changes recommendation selection and ranking and uses mean preference and NDCG@3 to support the final selection of `beta = 0.50`.
Recommendation explanation	MR / RS / CGS	Exposes the evidence behind a recommendation, links the selected output to its scores and constraints, and communicates the result in a user-understandable form.
Patch-based requirement refinement	MR / CGS	Represents additions, removals and replacements explicitly while maintaining the user's dialogue state and previously confirmed facts.
Selective pipeline re-execution	MR / RS	Reasons over dependencies between changed requirements and affected stages so that stale results are invalidated and only necessary processing is repeated.
Confirmation of the latest recommendation	MR / CGS	Maintains conversational state, validates the user's selection against the current reasoning round and prevents actions on obsolete outputs.

Implemented functionality	Course mapping	Application of course knowledge and techniques
Interactive explanation, refinement and confirmation controls	CGS	Supports human-centred communication and keeps the user in the decision loop instead of treating recommendation as a one-way prediction.
AI outfit preview with consent and fallback	CGS	Uses visual interaction to help the user understand a confirmed recommendation while preserving consent, privacy boundaries and graceful degradation.
Typed interfaces and cross-module hand-off validation	MR / RS	Preserves the semantics and provenance of inputs used by the reasoning and recommendation stages and rejects incomplete or inconsistent evidence.

Appendix C: Installation and User Guide

This appendix explains how to install and run WearTo locally and how to use the website. The application code is in `SystemCode/clips`; run all commands from that folder. Full module-level details are in `SystemCode/clips/docs/wearto-runbook.md`.

C.1 Requirements

Item	Requirement
Platform	macOS, Linux or Windows 10/11; Python **3.11** (installed by uv); Git; a modern browser.
Hardware	A CPU is sufficient (GPU optional). Several GB of disk space for models, embeddings and run outputs.
Network	Needed for the first model downloads and for live Open-Meteo, Unsplash, eBay, ECB and OpenAI calls.
Accounts	Unsplash Access Key; eBay **Production** keyset (App ID, Cert ID); OpenAI image API key (optional, for real AI previews). Open-Meteo needs no key.

Table C.1 System requirements

C.2 Installation

C.2.1 Install the Dependencies

Install Git and uv (https://docs.astral.sh/uv/), then clone the repository and install all website modules in one command:

```
git clone https://github.com/caiji666/IRS-PM-2026-10-25-AIS08FT-GRP17-WearTo.git
cd IRS-PM-2026-10-25-AIS08FT-GRP17-WearTo/SystemCode/clips
uv python install 3.11
uv sync --locked --extra products --extra inspiration --extra qwen --extra outfits --extra ebay-notifications --extra preview
```

Keep these extras together: a later plain `uv sync` removes them, so use `uv run --no-sync` for every subsequent command.

C.2.2 Download the Models and Train the Outfit Scorer

The website loads models offline, so download FashionCLIP and Qwen3-0.6B into `.cache/huggingface` first, then train the aligned Student 4 compatibility model (several minutes on a CPU; add `--device mps` on Apple Silicon):

```
uv run --no-sync python -m travel_agent.products.retrieval.m5.download --model-cache .cache/huggingface
uv run --no-sync python -c "from huggingface_hub import snapshot_download; from
travel_agent.understanding.extraction import MODEL_ID, MODEL_REVISION; snapshot_download(repo_id=MODEL_ID,
revision=MODEL_REVISION, cache_dir='.cache/huggingface')"
uv run --no-sync python -m travel_agent.outfits.train --reencode
```

C.2.3 Configure the Credentials

Credentials live only in Git-ignored local files. Copy `.env.example` to `.env` and create a `.secrets/` folder:

File	Contents
`.secrets/unsplash.env`	`UNSPLASH_ACCESS_KEY=<key>` (required by the live runner even if also in `.env`).
`.secrets/ebay-production.json`	`{"environment": "production", "client_id": "<App ID>", "client_secret": "<Cert ID>"}`. Sandbox keys are not accepted.
`.env`	`PREVIEW_PROVIDER=openai`, `PREVIEW_API_KEY`, `PREVIEW_MODEL` and `PREVIEW_ALLOWED_IMAGE_HOSTS=i.ebayimg.com,images.unsplash.com` (optional, for real AI previews).

Table C.2 Credential files

Check the eBay keyset with `uv run --no-sync python -m travel_agent.products.retrieval.ebay --credentials .secrets/ebay-production.json auth-check`. On macOS/Linux restrict the files with `chmod 600`. Restart the server after editing `.env`.

C.2.4 Verify the Installation (Optional)

`uv run --no-sync python -m travel_agent.demo` and `uv run --no-sync pytest -q` need no models, network or keys. To preload the 21 sample wardrobe photos, run `uv run --no-sync python -m travel_agent.products.retrieval.phase2 --offline ingest --images data/examples/wardrobe/images/*.png --output runs/wardrobe/baseline_import.json` (in PowerShell, replace the wildcard with `(Get-ChildItem data\examples\wardrobe\images\*.png).FullName`).

C.3 Launching the Website

```
uv run --no-sync python -m travel_agent.web --port 8765
```

Open **http://127.0.0.1:8765/** and keep the terminal running; press Ctrl+C to stop. The Profile page is at `/profile`. The server listens on the local computer only and must not be exposed through a public tunnel. Restarting it clears in-memory recommendation sessions but keeps saved results and wardrobe items.

C.4 User Guide

C.4.1 Profile and Wardrobe (Optional)

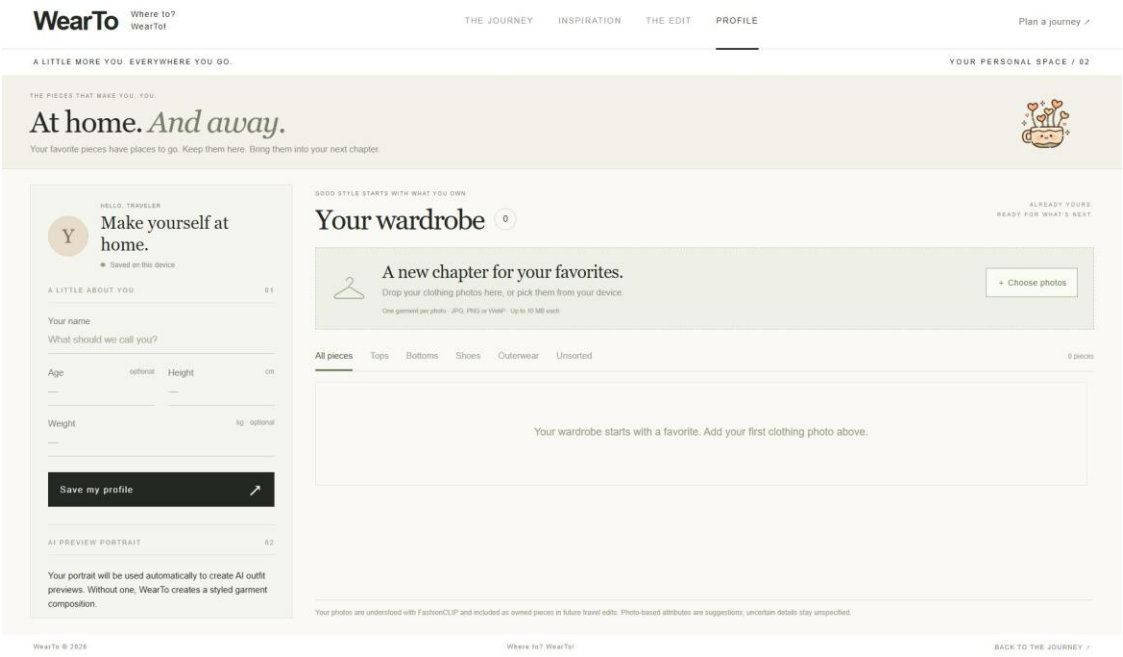


Figure C.1 Profile page: personal details and wardrobe upload

1. Open **PROFILE**, enter optional personal details and click **Save my profile**. They stay on this computer and do not determine clothing size.
2. Click **Upload or replace portrait** to use your photo in AI previews; without one, WearTo creates a flat-lay composition.
3. Click **Choose photos** (one garment per photo, JPG/PNG/WebP, at most 5 photos of up to 10 MB each per batch), then **Add to my wardrobe**. Items are sorted into **Tops**, **Bottoms**, **Shoes**, **Outerwear** or **Unsorted** and join every future journey.

C.4.2 Plan a Journey (UC01)

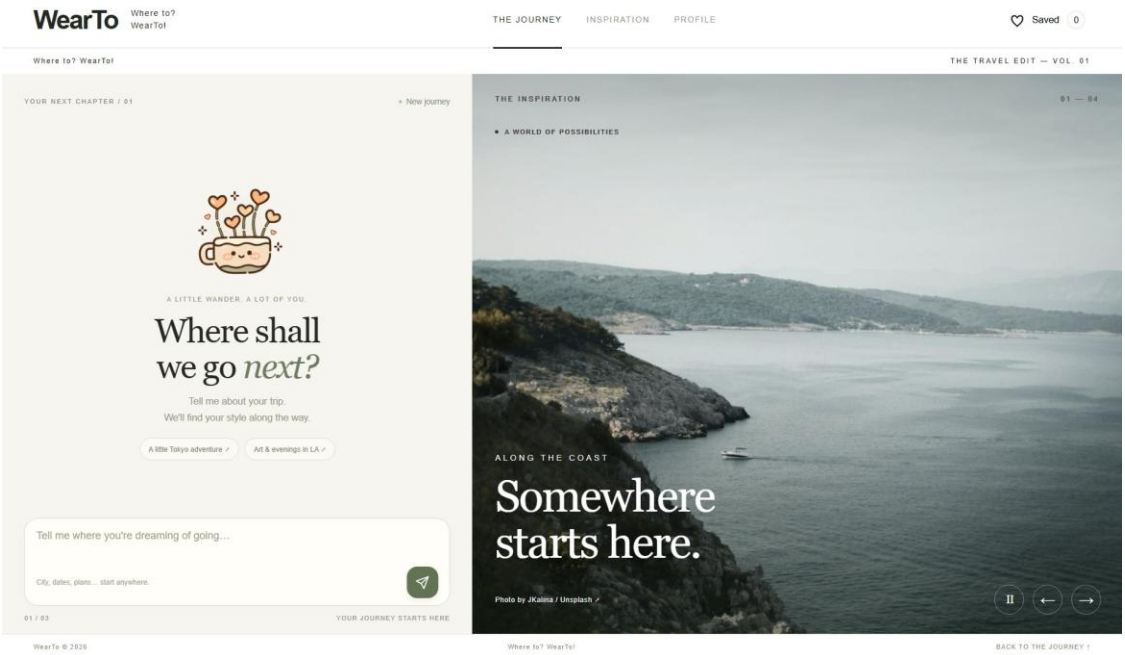


Figure C.2 Home page: travel chat and inspiration slideshow

1. Click **New journey** and describe the trip, e.g. `Seoul, 2026-12-20 to 2026-12-27. Sightseeing and dinner. Budget 500 USD. I prefer minimalist style.` Destination (a city), dates and budget with currency are required and can be given across several messages.
2. WearTo asks only for missing or unclear details. If activities or style are missing, select suggested bubbles and click **Continue**, or click **Skip** / **Skip all**.
3. Click **Review my trip**, check the details, optional dress code and **Confirm the destination**, then click **Confirm my trip & find pieces**. No search runs before this point.
4. Wait while the progress dialog moves through **Your plans**, **Weather**, **Inspiration**, **Products**, **Outfits** and **Top-3** (usually a few minutes).

C.4.3 View the Recommendations (UC02)

- **THE INSPIRATION** shows the ranked style reference photos for the destination.
- **YOUR FINAL THREE** shows the outfits as **OUTFIT 1–3** tabs with item photos, prices and stock labels; owned items are marked **YOUR WARDROBE**. Click a product for its eBay link, or the heart icon to save it.
- Fewer than three outfits, or a conflict message, means the budget or stock constraints removed the other candidates; constraints are never relaxed silently.

C.4.4 Refine the Recommendations (UC04)

Click **Adjust requirements** and type the change. Budget, size and item changes (`Change the budget to 300 USD`, `My size is M`, `I don't like the shoes from outfit 1`) are applied immediately to the existing outfits. Changes to the destination, dates, activities, style or currency open **Refresh your recommendations?**; click **Continue and update** to rerun the affected stages (Table C.3) or **Keep current recommendations** to cancel.

Changed requirement	Stages rerun
Budget, size, excluded items	10–12 (re-filter and re-rank only)
Currency	8–12
Style	4–12
Activities	3–12
Destination, dates	2–12

Table C.3 Selective re-execution after a requirement change

C.4.5 Confirm and Preview an Outfit (UC03)

1. Select an outfit tab and click **Confirm this outfit**; only outfits from the latest round can be confirmed.
2. Click **Generate AI preview** (usually 20–60 seconds). A saved portrait gives a portrait preview; otherwise a flat lay is produced. If the image provider is unavailable or not configured, the dialog shows "The preview image could not be generated." and the confirmed outfit stays on its product cards under **YOUR FINAL THREE**.
3. Click **Download image** to save the preview or **Generate again** for another version. Use **New journey** to start an unrelated trip; saved results are kept.

C.5 Troubleshooting

Symptom	Fix
Missing Unsplash configuration	Create `.secrets/unsplash.env`; a key only in `.env` is not enough.
eBay access error	Use an activated Production keyset (App ID → `client_id`, Cert ID → `client_secret`) and run `auth-check`.
Model unavailable offline	Complete the downloads in Section C.2.2 before starting the server.
`ModuleNotFoundError` after updating	Re-run the full `uv sync --locked --extra …` command.
"A few details need attention"	The trip needs clarification (e.g. a city or dress code); answer in the chat.
"Student 5 session expired"	The server restarted; reload the page and use **New journey**.
AI preview unavailable	Confirm a current outfit first; check the `PREVIEW_*` settings and OpenAI access; restart the server.

Table C.4 Common problems and fixes

References

[1] Chia, P. J., Attanasio, G., Bianchi, F., Terragni, S., Magalhães, A. R., Goncalves, D., Greco, C., and Tagliabue, J. (2022). Contrastive language and vision learning of general fashion concepts. Scientific Reports, 12, 18958. https://doi.org/10.1038/s41598-022-23052-9

[2] Shirkhani, S., Mokayed, H., Saini, R., and Chai, H. Y. (2023). Study of AI-driven fashion recommender systems. SN Computer Science, 4, Article 514. https://doi.org/10.1007/s42979-023-01932-9

[3] Jannach, D., Manzoor, A., Cai, W., and Chen, L. (2021). A survey on conversational recommender systems. ACM Computing Surveys, 54(5), Article 105. https://doi.org/10.1145/3453154

[4] Xiao, B., and Benbasat, I. (2018). An empirical examination of the influence of biased personalized product recommendations on consumers' decision making outcomes. Decision Support Systems, 110, 46-57. https://doi.org/10.1016/j.dss.2018.03.005

[5] Zhou, L., Wang, W., Xu, J. D., Liu, T., and Gu, J. (2018). Perceived information transparency in B2C e-commerce: An empirical investigation. Information & Management, 55(7), 912-927. https://doi.org/10.1016/j.im.2018.04.005